# Counting Fruits with State-of-the-Art Multi-View-Stereo Approaches

Bachelor Thesis in Computer Science


presented by

**Alexander Link**

born on 29.08.2004 in Bamberg


submitted to


**Department of Computer Science**
**Professorship for Cognitive Computer Vision**
**Friedrich-Alexander-Universität Erlangen-Nürnberg**

Supervisor: Lukas Meyer

Academic Supervision: Prof. Dr. Bernhard Egger


Begin date: 01.05.2026

End date: 01.10.2026

vce visual computing erlangen

# Abstract

Counting the fruit on trees from images is an important building block of precision agriculture. Counting in a 3D reconstruction of the tree aims to include each fruit only once, even though it appears in many images. Existing 3D Methods, however, require time-consuming optimization for every scene. In this thesis, we investigate whether the feed-forward multi-view stereo model VGGT, which predicts a reconstruction in a single forward pass, can replace this optimization. We develop a pipeline that associates apple instance masks across views through the predicted geometry and counts the resulting instances. The pipeline produces a count within a few minutes, most of which are spent on the segmentation, and correctly identifies every apple of a simple table scene. However, on a synthetic and a real apple tree the reconstruction is not accurate enough at the scale of individual apples, so that the association splits and merges apples and the count becomes unreliable. We attribute this to the small size of the apples in images capturing a whole tree, and outline how this limitation could be overcome.

# Contents


**1 Introduction** **1**
- 1.1 Challenges . . . 1
- 1.2 Our Approach . . . 2
- 1.3 Contributions . . . 2

**2 Related Work** **5**
- 2.1 Fruit Counting and Yield Estimation . . . 5
- 2.2 Multi-View Instance Association and 3D Segmentation . . . 6

**3 Fundamentals** **7**
- 3.1 VGGT . . . 7
  - 3.1.1 Context . . . 7
  - 3.1.2 Input Data . . . 8
  - 3.1.3 Pipeline . . . 8
  - 3.1.4 Output . . . 9
  - 3.1.5 Advantages and Limitations . . . 9
- 3.2 Graph Theory . . . 10
  - 3.2.1 Basic Concepts . . . 10
  - 3.2.2 Union-Find . . . 11
- 3.3 Hungarian Matching . . . 11
  - 3.3.1 The Assignment Problem . . . 11
  - 3.3.2 The Hungarian Algorithm . . . 12
  - 3.3.3 Partial Matching . . . 12
- 3.4 RANSAC . . . 13
- 3.5 Clustering . . . 13
  - 3.5.1 Density-Based Clustering . . . 13
  - 3.5.2 Agglomerative Clustering with Complete Linkage . . . 14

**4 Method** **17**
- 4.1 Overview . . . 17
- 4.2 Input: Reconstruction and Segmentation . . . 18
  - 4.2.1 3D Reconstruction . . . 18
  - 4.2.2 Instance Segmentation . . . 19

4.2.3 Outlier Removal . . . 19
4.3 Cross-View Mask Association . . . 19
4.3.1 Pixel Correspondence . . . 20
4.3.2 Mask Overlap and Matching Cost . . . 20
4.3.3 Pairwise Matching . . . 21
4.3.4 Aggregation into Multi-View Instances . . . 22
4.4 Apple Model: Sphere Fitting and Filtering . . . 22
4.5 Counting: Clustering of Sphere Centers . . . 24
4.6 Error Analysis of the Base Pipeline . . . 25
4.7 Iterative Refinement . . . 26
4.8 Implementation Details . . . 27

**5 Results** **29**
5.1 Experimental Setup . . . 29
5.1.1 Datasets . . . 29
5.1.2 Parameters . . . 30
5.2 Table Scene . . . 30
5.3 Synthetic Tree . . . 32
5.4 Real Tree . . . 34
5.5 Summary . . . 35

**6 Conclusions** **37**
6.1 Future Work . . . 38

# Chapter 1

# Introduction

Feeding a growing world population is a major global challenge, and projections estimate that total food demand will rise substantially over the coming decades [9]. Precision agriculture responds to this by using advanced technology and data analysis to maximize crop yields while reducing waste and environmental impact [20]. Within precision agriculture, counting the fruit on a tree is an important building block: it provides the yield estimates that growers rely on to plan the harvest and allocate labor, storage, and other resources [31]. Because performing the count manually across many trees is slow and labor-intensive, automating it from images is a valuable capability.

Among the ways to automate fruit counting, reconstructing the tree in 3D from a set of RGB images is particularly attractive, for two reasons. First, lifting the count into a shared three-dimensional representation lets each fruit be counted exactly once, instead of being counted repeatedly across the many overlapping images in which it appears. Second, capturing the necessary data requires only an ordinary color camera, of the kind built into any consumer device, rather than the specialized and more expensive depth-sensing equipment, such as LiDAR scanners or calibrated stereo rigs, on which other 3D-capture methods depend. This makes the approach inexpensive and easy to deploy in the field.

Recent work such as FruitNeRF [29] shows that image-based 3D fruit counting is feasible by training a Neural Radiance Field [30] per scene and counting the fruit in the resulting volumetric representation. This approach is effective, but it requires a time-consuming optimization for every new scene. In this work, we instead investigate whether a feed-forward multi-view stereo model, VGGT [40], can replace this per-scene training. Rather than optimizing a representation for each scene, VGGT predicts a 3D reconstruction in a single forward pass. This promises a significant reduction in runtime, but raises the central question of whether the resulting geometry is accurate enough for reliable counting.

Throughout this thesis, we focus on apples. Their nearly spherical shape allows for a simple geometric model, and FruitNeRF provides a real-world dataset of apple trees with manually counted ground truth [29], which enables a direct comparison with the optimization-based approach.

## 1.1 Challenges

Counting fruit reliably from images of a tree faces a number of difficulties, some inherent to the trees themselves and some arising from the way they must be captured.

The trees are difficult subjects in their own right. Fruit are frequently occluded, both by foliage

and by other fruit, so that any single image captures only part of the crop. Their appearance changes with the viewing angle and with lighting conditions such as direct sunlight or shadow, which makes the same fruit look different when seen from different viewpoints. In addition, the branches, leaves and fruit of a tree canopy form a geometrically complex, cluttered structure that is hard to reconstruct consistently.

The way the tree is captured adds further difficulties. To limit occlusion and avoid missing fruit, a tree should be observed from all sides. The number of images that can be used per tree is bounded, however, by the memory available to the reconstruction model (see Section 3.1.5). To still achieve full coverage, the cameras must therefore be placed sparsely around the tree, spanning a full 360-degree range. As a consequence, neighboring images are taken from widely spaced viewpoints and often show markedly different regions of the tree, with little visual content in common. This makes it hard to establish *correspondences* between images, that is, to determine which parts of one image show the same physical points as another. The problem is compounded by the scene not being perfectly static during capture: wind moves branches, leaves, and fruit between one image and the next, so that even points that are visible in both images no longer align, further degrading the cross-view correspondences on which the later counting steps rely. These unreliable correspondences are the root cause of the counting errors this thesis addresses: depending on the situation, the same apple may be split into several counts or several apples merged into one. How these errors arise in our pipeline, and how we counteract them, is discussed in Chapter 4.

## 1.2 Our Approach

To count the fruit in a scene, our method maps semantic information, in the form of 2D instance masks obtained with SAM3 [4], across views by means of the 3D representation predicted by VGGT. The association of masks across views follows the point-map-based principle introduced by CCGS [19]: pixel correspondences between views are established through the predicted point maps, and instance masks are matched across views by solving an assignment problem that allows partial matches. We adopt this association step, transfer it from the image-pair setting of its origin to the multi-view reconstruction produced by VGGT, and build a new counting pipeline on top of it. In this pipeline, the pairwise mask matches are aggregated into multi-view instances through a graph formulation, and the fruit are counted by combining these associated instances with a subsequent clustering step.

## 1.3 Contributions

This thesis makes the following contributions. We investigate whether the feed-forward multi-view stereo model VGGT can replace per-scene optimization for image-based fruit counting, and we assess whether its predicted geometry is accurate enough for this task. Building on the point-map-based mask association introduced by CCGS [19], we develop a fruit-counting pipeline that extends this association with a graph-based aggregation of instances across many 360-degree views, sphere fitting as an apple model, and a final clustering step to obtain the count. We further propose an iterative refinement of this pipeline that detects over-merged instances and re-examines them with stricter thresholds. Finally, we evaluate the pipeline on a table scene, a synthetic tree with full ground truth, and a real apple tree

from the FruitNeRF dataset, and trace its errors from the count back to the reconstruction.

# Chapter 2

# Related Work

## 2.1 Fruit Counting and Yield Estimation

Methods for counting fruit differ mainly in where the counting takes place: in the individual image, across a sequence of images, or in a three-dimensional representation of the scene. These stages can be read as increasingly robust answers to the core difficulty of counting across views, namely avoiding that the same fruit is counted more than once.

The earliest and simplest methods operate purely in the image plane, separating the task into detecting fruit in each image and counting the detections [15]. Detection has moved from handcrafted color and shape features to deep neural networks [31], and modern detectors such as YOLO reach high accuracy on individual images [1]. Because these methods count on a per-frame basis, however, they have no notion of which detections in different images correspond to the same fruit, so a fruit visible in several frames is counted several times.

To resolve this, a second line of work tracks fruit across consecutive frames. Liu et al. [25] proposed the first complete pipeline of this kind, using Structure-from-Motion (SfM) to link detections across frames and correct the resulting double counts, and recent pipelines pair modern detectors with dedicated tracking modules for the same purpose [5]. In later work, Liu et al. [26] matched detections between frames with the Hungarian algorithm on top of a semantic SfM reconstruction, using a single camera. What these methods share is a reliance on densely sampled video: consecutive frames differ only slightly, so detections can be tracked through strong frame-to-frame correspondences, and the association is typically driven by 2D image-space cues such as bounding-box overlap or optical flow between adjacent views. Our setting is the opposite. To cover a tree from all sides within a limited view budget, the cameras are placed sparsely around a full 360-degree range, so neighboring views often show markedly different regions of the tree and share little image-space overlap. Frame-to-frame tracking in the image plane is therefore not applicable, and the association must instead be established in 3D, through the reconstructed geometry rather than through 2D overlap. We adopt the Hungarian algorithm for this association as well, but apply it to correspondences derived from the 3D reconstruction rather than 2D bounding-box overlap between adjacent frames.

A third line of work removes double counting structurally by moving the count into three dimensions, so that each fruit exists once in a shared spatial representation. Häni et al. [15] reconstructed rows of trees from both sides and merged the counts from the combined 3D models. FruitNeRF [29] takes

this idea furthest by integrating the counting directly into a neural scene representation, unifying reconstruction and segmentation that earlier methods had kept separate. Its drawback is that the neural representation must be optimized for each scene, which is computationally expensive; this cost is the immediate starting point of the present work. Newer methods reduce it by replacing the neural field with faster representations, such as the 3D Gaussian splatting used by FruitGaussian [18] and CountingFruit [24], but they remain tied to a per-scene optimization. A different way to avoid this optimization is to associate instance masks across views through a 3D representation rather than to optimize one; since this is the strategy of the present work, we treat it separately in the next section.

## 2.2 Multi-View Instance Association and 3D Segmentation

Associating 2D instance masks across many views into consistent 3D instances is a problem closely related to fruit counting, and one that general-purpose 3D scene-understanding methods address independently of any counting application. What distinguishes these methods is the criterion by which they decide that two masks seen in different views depict the same object.

The most direct criterion is geometric overlap in an explicit reconstruction. Gené-Molá et al. [14] segment fruit with Mask R-CNN, project the 2D masks into an SfM point cloud, associate detections by their overlap in 3D, and discard false detections with a support vector machine. The association is accurate but inherits the cost of the underlying SfM reconstruction, which the authors note is computationally intensive and unsuitable for real-time use.

A second criterion replaces this geometric overlap with cross-view agreement. MaskClustering [43] builds a global graph whose nodes are the 2D masks and weights each edge by a *view consensus rate*, the fraction of masks from other views that contain both endpoints; instances then emerge from iteratively clustering the masks that agree most strongly. The decision is thereby made globally, over all views at once, rather than only between neighboring frames, but it rests on how masks co-occur across views rather than on explicit 3D correspondences.

A third criterion is learned from the features of a reconstruction model. VGGT-Segmentor [13] trains a segmentation head on top of the frozen VGGT encoder that, given the mask of an object in one view, predicts the mask of the same object in a second view. Its authors observe that under large changes of viewpoint, the pixel-level correspondences of VGGT drift, while its internal features remain aligned at the level of objects. The method transfers a single query mask between a pair of views and relies on a trained head.

The criterion our method builds on, introduced by CCGS [19], derives the association directly from dense point maps. Pixel correspondences between views are found by minimizing the Euclidean distance between the predicted point maps; the overlap between two masks is redefined in terms of these correspondences; and the resulting matching cost is solved with the Hungarian algorithm, with partial matches permitted. CCGS uses the associated masks to build a compact segmentation field for 3D Gaussian splatting, and it operates on image pairs from a DUSt3R reconstruction. We take this point-map-based association and, instead, apply it to the joint multi-view reconstruction produced by VGGT, using it as the first stage of our counting pipeline.

# Chapter 3

# Fundamentals

## 3.1 VGGT

The central component of the reconstruction stage in this work is VGGT (Visual Geometry Grounded Transformer), a feed-forward transformer introduced by Wang et al. [40] and awarded the Best Paper Award at CVPR 2025. In this section, we first contextualize VGGT within the broader field of 3D reconstruction and then examine its input data, internal pipeline, and outputs. Finally, we discuss the properties that make it a suitable foundation for our counting method.

### 3.1.1 Context

Reconstructing the three-dimensional structure of a scene from a set of two-dimensional images is a long-standing problem in computer vision [16]. The classical solution is a multi-stage pipeline consisting of Structure-from-Motion (SfM) and Multi-View Stereo (MVS). In the implementation by Schönberger and Frahm [36], SfM estimates the camera poses as well as a sparsely populated point cloud by extracting and matching handcrafted features across the images, followed by an iterative optimization process known as bundle adjustment. A subsequent MVS step then densifies this reconstruction. While these methods are accurate and remain widely used, they rely on iterative optimization that is computationally expensive and can take several minutes to hours per scene to complete.

More recently, learning-based approaches have replaced parts of this pipeline with neural networks. Neural Radiance Fields (NeRFs) [30] represent a scene as a volumetric radiance function encoded in the weights of a neural network, while 3D Gaussian Splatting [21] models it as a collection of colored Gaussian distributions. Both produce high-quality reconstructions but require separate optimization for each individual scene, which again incurs a substantial runtime cost per reconstruction.

A different line of work aims to eliminate per-scene optimization entirely by predicting geometry with a neural network. DUSt3R [41] pioneered this *feed-forward* principle by regressing a dense point map from an image pair in a single network pass, followed by MASt3R [23], which added feature matching. However, these methods operate on image pairs and still require a global alignment step to fuse the pairwise predictions into a consistent scene. VGGT [40] belongs to this feed-forward family, but generalizes it from pairs to arbitrary sets of images: it processes one, a few, or up to hundreds of views together in a single forward pass and predicts a globally consistent reconstruction without any

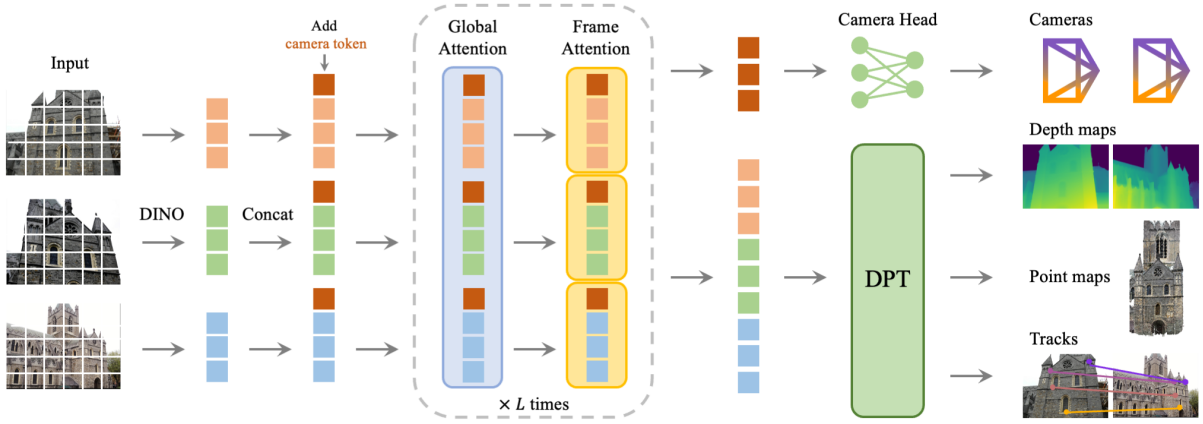



**Figure 3.1:** Overview of the VGGT architecture, taken from [40]. The input images are split into patch tokens by DINO and augmented with camera tokens. $L = 24$ blocks of frame-wise and global self-attention process the tokens of all images jointly, after which the camera head predicts the camera parameters and the DPT head the dense outputs.

subsequent optimization. This combination of speed and multi-view consistency motivates its use in this work, where fast reconstruction is a central goal.

### 3.1.2 Input Data

The input for VGGT is a sequence of $N$ RGB images $(I_i)_{i=1}^N$ with $I_i \in \mathbb{R}^{3\times H\times W}$, which capture the same static scene. Unlike the classical SfM pipeline, no camera parameters, calibration data, or prior pose information need to be provided: the model derives all geometric quantities directly from the image content. The number of views $N$ is flexible and can range from a single image to several hundred, with the only practical limitation being the available GPU memory, since all views are processed jointly [40].

### 3.1.3 Pipeline

Internally, VGGT maps the input sequence through three conceptual stages to a set of dense 3D predictions: tokenization, alternating attention, and task-specific prediction heads. An overview is shown in Figure 3.1.

**Tokenization.** Each input image $I_i$ is first divided into a grid of non-overlapping image patches. Each patch is then encoded into a *token*, a fixed-length vector that represents the local content of that patch and serves as the basic unit on which the network operates [10]. In VGGT, this patch encoding is performed by a pretrained DINOv2 backbone [33]. An image is thus converted into a set of tokens, each of which carries the local appearance and structural information of its region. For each image, the patch tokens are augmented with additional learnable tokens: a *camera token*, which aggregates the information needed to predict the camera parameters of this image, as well as a set of *register tokens*, which serve as auxiliary memory during the attention calculation and are discarded before the final prediction.

**Alternating Attention.** The core of the network is a transformer [38] that processes all tokens of all images jointly. It consists of 24 attention blocks, each of which applies frame-wise and global self-attention in turn. In *frame-wise* self-attention, tokens attend only to other tokens of the same image, which preserves and refines the local structure of each image. In *global* self-attention, all tokens attend to one another across all images, thereby exchanging information between views and establishing the correspondences that link the individual images into a single, coherent scene. By alternating between these two modes, the network balances local detail with global multi-view consistency. This alternating attention design is the central architectural mechanism of VGGT; the authors show that it is superior to both purely global attention and cross-attention alternatives [40].

**Prediction Heads.** After the attention blocks, the refined tokens are passed to specialized prediction heads. A dedicated *camera head* processes the camera tokens to regress the intrinsic and extrinsic camera parameters of each view. A *DPT head* [35], a Dense Prediction Transformer, upsamples the image tokens back to the input resolution to generate the dense, pixel-wise outputs, namely depth maps, point maps, tracking features, and the corresponding confidence maps.

### 3.1.4 Output

For each input image $I_i$, VGGT predicts the following quantities:

- camera extrinsics and intrinsics of the view,
- a dense depth map together with a per-pixel confidence map,
- a dense point map together with a per-pixel confidence map, and
- dense feature maps used for tracking pixels across views.

The point map is the output most relevant to this work. It assigns a 3D scene coordinate $P_i(y) \in \mathbb{R}^3$ to every pixel $y$ of an image, thereby representing a dense, pixel-wise 3D reconstruction. Crucially, the point maps of all images are expressed in a single, common coordinate system: they are defined in the reference frame of the first camera, which serves as the world origin [40]. Points observed in different views therefore already lie in a common 3D space without requiring an explicit alignment or registration step. This shared coordinate frame is the property our method builds upon, as it allows 2D instance masks from different views to be related to one another purely through their 3D coordinates.

Each dense prediction is accompanied by a confidence map that estimates the pixel-wise reliability of the reconstruction. These confidence values provide a signal for identifying and filtering out unreliable regions, such as poorly observed or geometrically ambiguous areas [40].

### 3.1.5 Advantages and Limitations

The main advantage of VGGT for our application is its speed. Since the entire reconstruction is generated in a single forward pass, a scene can be reconstructed in a matter of seconds, whereas the SfM-based and scene-specific optimization approaches discussed in Section 3.1.1 take minutes to hours. At the same time, according to the authors, VGGT matches or exceeds the accuracy of these optimization-based alternatives across multiple 3D tasks [40], and does so without any post-processing. Another practical

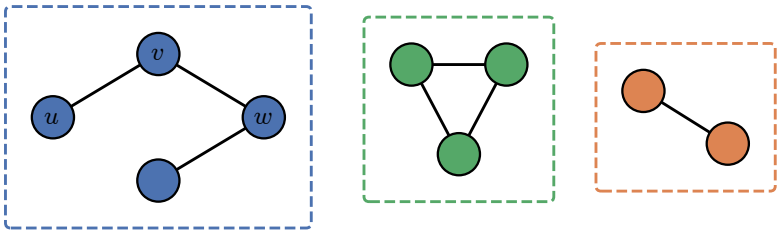



**Figure 3.2:** A graph with three connected components (dashed). The nodes $u$ and $w$ are not adjacent, but they belong to the same component because they are connected by a path via $v$.

advantage is that it operates directly on uncalibrated images and outputs camera parameters, depth, and point maps together, so that no separate calibration or pose estimation step is required.

The fundamental limitation is that all input views are processed collectively via global self-attention, whose memory requirements increase with the number of tokens and, consequently, with the number of input views. The authors note that a naive implementation can become very memory intensive when dealing with a large number of tokens, so that in practice, the available GPU memory limits the number of images that can be reconstructed together [40]. This constraint is directly relevant to our scenario, in which a fruit tree must be captured from all sides: The limited view budget forces a sparse distribution of cameras around the scene, which in turn weakens the correspondences between neighboring views. A number of follow-up works address this scaling limit, for example by processing long sequences in segments, as in VGGT-Long [8] and VGGT-SLAM [27], or by scaling up to thousands of images as in Fast3R [44]; Pi3 [42] additionally eliminates the dependence on a fixed reference view. These extensions are beyond the scope of this work, which uses the original VGGT model.

## 3.2 Graph Theory

This section introduces the graph theory concepts used in Chapter 4: connected components and the Union-Find data structure, which computes them efficiently.

### 3.2.1 Basic Concepts

A graph $G = (V, E)$ consists of a set of vertices (or nodes) $V$ and a set of edges $E \subseteq V \times V$, where each edge connects a pair of vertices. Two vertices are said to be *adjacent* if they are connected by an edge. A *path* is a sequence of vertices in which every consecutive pair of vertices is connected by an edge. A *connected component* is a maximal subset of vertices in which every pair of vertices is connected by a path and which is not connected to any vertex outside the subset. Intuitively, a connected component is a group of nodes that are all directly or indirectly connected to one another, while being separated from the rest of the graph.

Connectivity is transitive: If node $u$ is connected to $v$ via a path and $v$ is connected to $w$, then $u$ is also connected to $w$. Thus, a connected component unites nodes that were never directly connected by an edge, as long as they are connected by a chain of edges. Figure 3.2 illustrates this for a small graph.

### 3.2.2 Union-Find

Computing the connected components of a graph via a disjoint-set data structure is a standard technique [6]. We use the *Union-Find* data structure, also known as the disjoint-set data structure, which manages a partition of the nodes into disjoint sets and efficiently supports two operations: `find`, which returns a representative element for the set containing a given node, and `union`, which merges the two sets containing two given nodes into one.

Each set is stored as a tree, in which every node points to a parent node, and the root of the tree serves as the representative of the set. The `find` operation follows the parent pointers from a node up to the root. Two nodes belong to the same set if and only if the `find` operation returns the same root for both. The `union` operation joins two sets by attaching the root of one tree as a child of the root of the other.

To compute the connected components within a graph, the Union-Find structure is initialized with a singleton set per node, and `union` is called for the two endpoints of every edge. After all edges have been processed, the sets of the Union-Find structure are exactly the connected components of the graph [6].

A naive implementation of `find` can degenerate when the trees become tall, since each call then traverses a long chain of parent pointers. To avoid this, *path compression* is applied: During a `find` operation, every node visited on the path to the root is immediately made to point directly to the root [6]. This makes repeated lookups on the same set progressively cheaper and keeps the extraction of connected components efficient even for large graphs.

## 3.3 Hungarian Matching

The assignment problem asks for an optimal one-to-one pairing between the elements of two sets. This section introduces the problem, the Hungarian algorithm [22, 32], which solves it exactly, and partial matching, which allows elements to remain unassigned.

### 3.3.1 The Assignment Problem

The assignment problem is a classical combinatorial optimization problem. In its original formulation [22], a set of $n$ workers is to be assigned $n$ jobs, given a numerical rating for the performance of each worker on each job, such that the total rating of the chosen assignment is optimal and no worker is assigned more than one job. More generally, the elements of two sets are paired, and each candidate pair carries a cost.

Let $A = \{1, \dots, n_A\}$ and $B = \{1, \dots, n_B\}$ be two sets with elements $a \in A$ and $b \in B$. For every pair $(a, b)$, a cost $C_{ab} \geq 0$ expresses how poorly the two elements fit together, where a lower cost indicates a better pair. These costs form a cost matrix $C \in \mathbb{R}^{n_A \times n_B}$.

In its standard form, the linear sum assignment problem assumes an equal number of elements on both sides and assigns each element exactly once. Describing an assignment by binary variables $x_{ab} \in \{0, 1\}$, where $x_{ab} = 1$ means that $a$ is matched to $b$, it is written with equality constraints as $\sum_b x_{ab} = 1$ and $\sum_a x_{ab} = 1$, so that every element receives exactly one partner [3]. If the two sets differ in size, this is not possible. Assuming $n_A \leq n_B$ without loss of generality, the rectangular form of the

problem assigns every element of the smaller set exactly once and every element of the larger set at most once:

$$\begin{aligned} \min_{x} \quad & \sum_{a} \sum_{b} C_{ab}\, x_{ab} \\ \text{s.t.} \quad & \sum_{b} x_{ab} = 1 \quad \forall a, \\ & \sum_{a} x_{ab} \leq 1 \quad \forall b, \\ & x_{ab} \in \{0, 1\}. \end{aligned} \tag{3.1}$$

Because elements of $A$ are only ever paired with elements of $B$, and never with elements of their own set, the underlying structure is a *bipartite* graph: its nodes split into the two disjoint sets, and every candidate match connects one set to the other, with no edges inside either set. The cost matrix $C$ is the weighted adjacency matrix of this bipartite graph, and the assignment problem is the search for a minimum-cost bipartite matching.

Because of this formulation, $n_B - n_A$ elements of the larger set always remain unassigned. Conversely, every element of the smaller set is assigned a partner, even if none of the candidates fit well. Such assignments are subsequently removed through partial matching (Section 3.3.3).

### 3.3.2 The Hungarian Algorithm

The difficulty of the assignment problem is that the number of possible assignments grows factorially with the number of elements: for two sets with $n$ elements each, there are $n!$ distinct one-to-one assignments [32], so enumerating them and picking the cheapest is infeasible for all but the smallest cases. A tempting shortcut is a greedy strategy in which each element is assigned to its individually best partner. Such a choice is locally optimal but can be globally suboptimal: two elements of $A$ might both prefer the same element of $B$, which violates the one-to-one requirement, and resolving the conflict greedily can force a worse overall assignment than necessary.

The Hungarian algorithm avoids both problems. Introduced by Kuhn [22], it is named for the underlying results of the Hungarian mathematicians D. König and E. Egerváry on which it is based, and it computes a globally optimal assignment without enumerating all possibilities. Munkres [32] later gave a variant of the method together with a proof that it terminates after a number of operations bounded by a polynomial in $n$, rather than the factorial cost of exhaustive search; because of this joint contribution the method is also known as the Kuhn-Munkres algorithm. It therefore solves the problem in Equation (3.1) exactly and efficiently.

Later algorithms, like the one by Jonker and Volgenant, solve the same problem but with lower runtime [7]. Since they all compute an optimal assignment, the choice between them affects only the runtime, not the result.

### 3.3.3 Partial Matching

The optimal assignment minimizes the total cost, but it does not by itself guarantee that the matched pairs are meaningful: the algorithm will continue to pair two poorly fitting elements if doing so reduces the total cost. In a *partial matching*, elements may instead remain unassigned when no suitable partner

exists. By first solving the assignment problem and then discarding all assigned pairs whose cost exceeds a threshold, we obtain such a matching. The remaining pairs are part of an optimal assignment and, in addition, individually acceptable.

## 3.4 RANSAC

Fitting a geometric model to a set of measured points is a common task in computer vision. Whenever such measurements originate from imperfect detectors or reconstructions, they contain not only small measurement noise but also *gross errors*: points that are simply wrong rather than slightly inaccurate. A conventional estimator such as least squares offers no protection against these. As an averaging technique, it relies on the assumption that any deviation from the model is a small measurement error and that such deviations balance each other out over a sufficiently large number of points [12]. Gross errors clearly violate this assumption, and even a single severely misplaced point can distort the entire fit. To robustly estimate a model under such conditions, we use RANSAC (Random Sample Consensus).

RANSAC was introduced by Fischler and Bolles [12] and reverses the logic of conventional smoothing: Instead of fitting the model to all points and then attempting to remove those that do not match, it fits the model to a small sample and then measures how many of the remaining points match it. Each iteration proceeds in three steps. First, it draws a *minimal sample*, the smallest number of points that suffices to instantiate the model, and computes the model from this sample alone. Second, it tests the instantiated model against all other points and collects every point that lies within a fixed error tolerance of it into the *consensus set* for this sample; points outside the tolerance are considered outliers. Third, the size of the consensus set is recorded as the value for this sample. These steps are repeated over a specified number of iterations, and the model with the largest consensus set is retained. Optionally, this final model is refitted to all points in its consensus set to refine the estimate [12].

The robustness of the method stems from this construction. Since the model is selected by consensus among many points rather than by minimizing a residual across all points, gross outliers that would derail an averaging estimator are excluded from the consensus set and therefore do not influence the retained model. Figure 3.3 illustrates this on the simple case of fitting a circle to a set of 2D points: three points determine a candidate circle, and the points within the tolerance band form its consensus set, while the outliers that would bias a least-squares fit are left out.

## 3.5 Clustering

Clustering divides a set of points into groups, or clusters, so that points that are far apart belong to different clusters, while points that are close to each other belong to the same cluster. First, we will discuss density-based clustering and then turn to the agglomerative method, which is better suited for spatially compact clusters.

### 3.5.1 Density-Based Clustering

A widely used density-based clustering algorithm is DBSCAN, introduced by Ester et al. [11]. Its central idea is that clusters are regions of high point density separated from one another by regions of low density. Density is evaluated based on a neighborhood radius $\varepsilon$ and a minimum number *MinPts*: A point

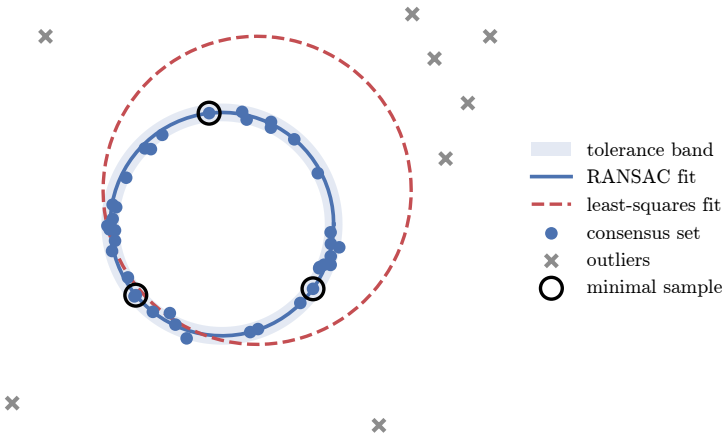



**Figure 3.3:** RANSAC fitting a circle to 2D points with outliers. Three points (minimal sample) determine a candidate circle, and the points within the tolerance band form its consensus set. The least-squares fit to all points (dashed) is pulled toward the outliers.

is a *core point* if at least *MinPts* points lie within a distance of $\varepsilon$ from it [11]. For two-dimensional data, the authors fix *MinPts* to a constant value, so that only $\varepsilon$ remains to be chosen [11].

Clusters are then formed from core points based on a concept of *density-reachability*: A point is density-reachable from a core point if it can be connected to it by a chain of points in which each successive point lies within $\varepsilon$ of the previous one and each point except possibly the last is itself a core point [11]. A cluster is a maximal set of density-connected points, and points that do not belong to any cluster are referred to as noise.

This chaining allows DBSCAN to discover clusters of any shape, but it can be a disadvantage if clusters are expected to be spatially compact. Since membership propagates transitively along chains of $\varepsilon$-close points, two dense groups are kept separate from each other only if the distance between them is greater than $\varepsilon$ [11]. If such groups are close to one another, the points of one may lie within $\varepsilon$ of the other, and DBSCAN then connects them via the intervening points to a single cluster. Different groups are thereby linked into a single one. Reducing $\varepsilon$ to break such chains is not a solution: it also separates loosely connected points within a single group, thereby splitting a cluster into several. In general, there is no single radius that separates neighboring groups while simultaneously uniting the points of each individual group.

### 3.5.2 Agglomerative Clustering with Complete Linkage

We therefore use agglomerative hierarchical clustering, which controls the spatial extent of a cluster directly rather than via a density chain. The method begins with each point in its own cluster and repeatedly merges the two nearest clusters [17, pp. 520, 523], where the distance between two clusters is defined by a *linkage* criterion. Under *complete linkage*, this distance is the greatest distance between any pair of points, one from each cluster [17, p. 523]; two clusters are thus merged only if *all* of their points are close to one another. The merging process ends as soon as the smallest distance between the clusters exceeds a specified threshold [17, p. 521].

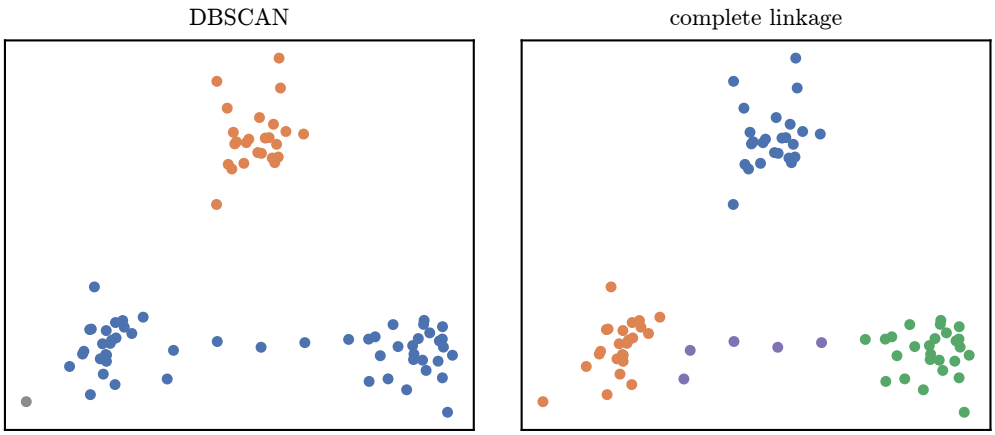



**Figure 3.4:** DBSCAN and agglomerative clustering with complete linkage on three compact groups, two of which are connected by a chain of points. DBSCAN links these two groups into a single cluster (left; gray: noise). Complete linkage keeps all three groups apart, and the chain forms a small cluster of its own (right).

Since complete linkage limits the maximum distance within a cluster, it also limits the diameter that a cluster can reach and tends to produce compact clusters [17, p. 524]. It therefore cannot link a sequence of points into a group of arbitrary length. This behavior, known as *chaining* [17, p. 524], is what affects density-based clustering as described above. Choosing the threshold according to the expected size of a group makes it possible to merge the points of a group while separate groups remain distinct from one another, so that each resulting cluster corresponds exactly to one group. Figure 3.4 contrasts both methods on three compact groups, two of which are connected by a thin chain of points.

# Chapter 4

# Method

## 4.1 Overview

Our method takes as input a set of $N$ RGB images $(I_i)_{i=1}^N$ of a tree, which were captured from different viewpoints around the tree, and returns the number of apples on the tree. As for VGGT itself (Section 3.1.2), no camera parameters are required. The method combines two complementary sources of information: VGGT provides the geometry by mapping each pixel in each view to the shared 3D coordinate system, while SAM3 [4] provides the semantics by segmenting individual apples in each view as 2D instance masks. However, the instance identifiers generated by SAM3 are meaningful only in a single view. It is not known which masks in the different views represent the same apple, and restoring these correspondences forms the core of the counting task.

We formulate this task on the graph introduced in Section 3.2.1. Its nodes are all instance masks $(i, m)$ across all views, and the goal is to divide these into groups so that each group contains exactly those masks that represent a single physical apple. The number of groups then corresponds to the count. Two opposite types of errors can occur: a *split error* distributes the masks of one apple over several groups and leads to overcounting, while a *merge error* combines the masks of several apples in one group and leads to undercounting. Every step included in the pipeline was designed with these two errors in mind.

Figure 4.1 provides an overview of the pipeline. After determining the point maps and instance masks of all views (Section 4.2), we match the masks between each pair of views and aggregate the pairwise matches into multi-view instances (Section 4.3). Each resulting instance is checked against a geometric model of an apple, a sphere (Section 4.4), and the centers of the accepted spheres are clustered to obtain the final count (Section 4.5). Section 4.6 analyzes the errors in this pipeline, and Section 4.7 describes an iterative refinement that addresses these errors. We refer to the pipeline without this refinement as the *base pipeline* and to the pipeline including it as the *refined pipeline*. Further details on the implementation can be found in Section 4.8.

The mask association between two views, i.e., the pixel correspondences derived from the distances in the point map, as well as the matching using partial assignments, follows CCGS [19]. Aggregation over many views, the apple model, clustering, and iterative refinement are contributions of this work.

To illustrate each individual step, we use a small scene as an example (Figure 4.2) throughout this chapter. It consists of seven apples lying on a table and partially separated by a cardboard partition, such

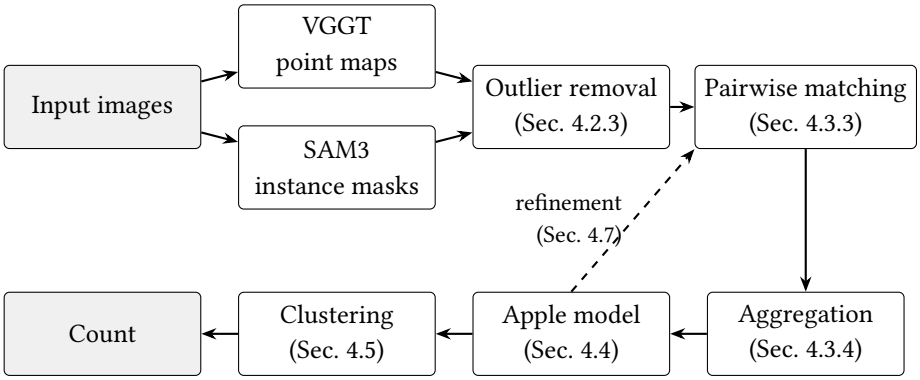



**Figure 4.1:** Overview of the counting pipeline. VGGT predicts a point map for every view, and SAM3 segments the apples as instance masks. After outlier removal, the masks are matched between all pairs of views and aggregated into multi-view instances, which are checked against a sphere as apple model. In the refined pipeline, instances that are too large are re-associated with stricter thresholds (dashed). The centers of the accepted spheres are clustered to obtain the count.

**Figure 4.2:** Running example used throughout this chapter: five views of seven apples on a table, partially separated by a cardboard partition.

that some apples are obscured from certain viewpoints. The scene was captured with a smartphone camera in five individual frames. Thanks to its small size, we can visualize intermediate results such as cost matrices and graphs, while the partition simulates, on a very small scale, the occlusion that makes association difficult with real trees.

## 4.2 Input: Reconstruction and Segmentation

### 4.2.1 3D Reconstruction

All $N$ images are processed together using VGGT in a single forward pass. We directly use the output of the point map head (Section 3.1.4) as the point map $P_i$ of view $i$. Wang et al. report that unprojecting the predicted depth maps using the predicted camera parameters yields more accurate points than the point map head [40]. In our setting with sparse 360-degree capture, we observe the opposite: The unprojected points from different views align less well with each other, whereas the point map head places the same surface point observed in different views almost at the same location (Section 5.3). Since the association relies entirely on spatial accuracy between views, we use the point map head. Its predictions are expressed in the coordinate system of the first view, so the property described in Section 3.1.4 is applicable. VGGT normalizes the scale of each reconstruction, so all distances and thresholds in this work are given in the units of the respective reconstruction rather than in meters.

### 4.2.2 Instance Segmentation

Each view is segmented independently using SAM3 [4]. Given the text prompt “apple”, SAM3 detects the instances of this concept in the image and outputs a mask for each of them. Masks whose area is smaller than a minimum area threshold are excluded to avoid small false positives. Each detected mask receives an identifier $m$ that is unique within its view.

SAM3 operates at the original image resolution, while the point maps are predicted at a lower VGGT input resolution. Therefore, we store the masks of each view as a single image consisting of instance identifiers and resize it to match the resolution of the point maps using nearest-neighbor interpolation. Unlike bilinear interpolation, this method never generates intermediate values that would confuse the identifiers of two neighboring instances. Thus, each pixel $y$ of a mask corresponds to exactly one point $P_i(y)$. We denote by $\mathcal{M}_{i,m}$ the set of pixels of the instance $m$ in view $i$, while $\mathcal{A}_i = \bigcup_m \mathcal{M}_{i,m}$ denotes the entire set of apple pixels in view $i$.

### 4.2.3 Outlier Removal

The predicted apple points are not error-free. In addition to minor deviations, they can contain isolated points that lie far from any apple surface, in particular at the edges of a mask, where the prediction changes between the apple and the background. Such points cause problems in two ways: they can lead to correspondences between the masks of two different apples (Section 4.3) and also enlarge the sphere fitted to the instance (Section 4.4).

Therefore, before the association, we remove the outliers from the apple points. The points $P_i(y)$ with $y \in \mathcal{A}_i$ from all views are combined into a single set of points, to which we apply radius outlier removal as implemented in Open3D [45]: a point is kept only if there are at least $k_\mathrm{f}$ points within its neighborhood of radius $r_\mathrm{f}$. Since the set is combined from all views, a point on the apple’s surface that was observed in multiple views is supported by points from other views, while an isolated point without such support is removed. The discarded points are not deleted, but rather marked as invalid in the point map so that the correspondence between pixels and points is preserved and invalid pixels are ignored in all subsequent steps. Only apple pixels are filtered, since all later steps are applied exclusively to these.

The confidence maps, which VGGT predicts in addition to the point maps (Section 3.1.4), could serve as an alternative filtering criterion. However, a confidence threshold that removes the same share of points as the radius-based filter yields a similar count, but locates the apples less accurately (Section 5.3). We therefore use the radius-based filter, which evaluates each point based on the geometry of its surroundings in all views.

## 4.3 Cross-View Mask Association

This section describes how the instance masks of all views are fused into multi-view instances. The process consists of two steps. For each pair of views, the masks are matched using the point-map-based association method from CCGS [19] (Sections 4.3.1 to 4.3.3). Subsequently, the pairwise matches are aggregated over all pairs to form the connected components of the graph from Section 3.2.1 (Section 4.3.4).

CCGS associates masks between adjacent frames of a densely sampled sequence, in which consecutive views show largely the same content. We adapt it to the sparse 360-degree setting in three ways: correspondences are established only between apple pixels, they are subject to a distance threshold, and the matching is performed for all view pairs, not just adjacent ones.

### 4.3.1 Pixel Correspondence

Based on CCGS, we determine the pixel correspondence between two views $i$ and $j$ using their point maps: a pixel in view $i$ corresponds to the pixel in view $j$ whose 3D point is closest to its own [19]. Since only the apple pixels are relevant for the correspondence, we only search among these. For a pixel $y \in \mathcal{A}_i$, a candidate correspondence is given by

$$\phi_{ij}(y) = \arg\min_{y' \in \mathcal{A}_j} \|P_i(y) - P_j(y')\|_2. \tag{4.1}$$

As described in Section 4.2.3, only valid pixels are considered. In CCGS, each pixel is assigned a correspondence. This is suitable for adjacent images in which almost every point is visible in both images, which is not the case here, however. An apple that is visible in view $i$ but obscured in view $j$ would still be assigned to the nearest apple in view $j$, regardless of how far away it is, and all of its pixels would end up in the same incorrect mask. Therefore, we accept a correspondence only if

$$\|P_i(y) - P_j(\phi_{ij}(y))\|_2 < \tau_{\text{corr}}, \tag{4.2}$$

otherwise we leave $y$ without a correspondence. The threshold $\tau_{\text{corr}}$ must account for the deviation between two predictions of the same surface point from different viewpoints, including the displacement caused by slight dynamic changes to the scene (Section 1.1). At the same time, it must remain small compared to the size of an apple. Near the point of contact between two adjacent apples, some correspondences between different apples are unavoidable, even if $\tau_{\text{corr}}$ is small. However, these affect only a few pixels, and are handled at the mask level (Section 4.3.3).

The correspondence is directed from view $i$ to view $j$ and is calculated once for each pair of views. Figure 4.3 illustrates this using two views from our example.

### 4.3.2 Mask Overlap and Matching Cost

CCGS defines the overlap of two masks in different views based on this correspondence [19]. For a mask $a$ of view $i$ and a mask $b$ of view $j$, their intersection consists of those pixels of $a$ whose correspondence lies in $b$:

$$\mathcal{M}_{i,a} \cap_\phi \mathcal{M}_{j,b} = \{\, y \in \mathcal{M}_{i,a} \,:\, \phi_{ij}(y) \in \mathcal{M}_{j,b} \,\}, \tag{4.3}$$

where pixels without an accepted correspondence are never considered. The overlap is the size of this intersection relative to the smaller of the two masks, while the matching cost is its complement:

$$o_{ab} = \min\left(1,\ \frac{|\mathcal{M}_{i,a} \cap_\phi \mathcal{M}_{j,b}|}{\min(|\mathcal{M}_{i,a}|, |\mathcal{M}_{j,b}|)}\right), \qquad C_{ab} = 1 - o_{ab}. \tag{4.4}$$

Normalization based on the smaller mask corresponds to the CCGS method and is suitable for partial visibility. If an apple is partially obscured in one view, its smaller mask may still completely overlap

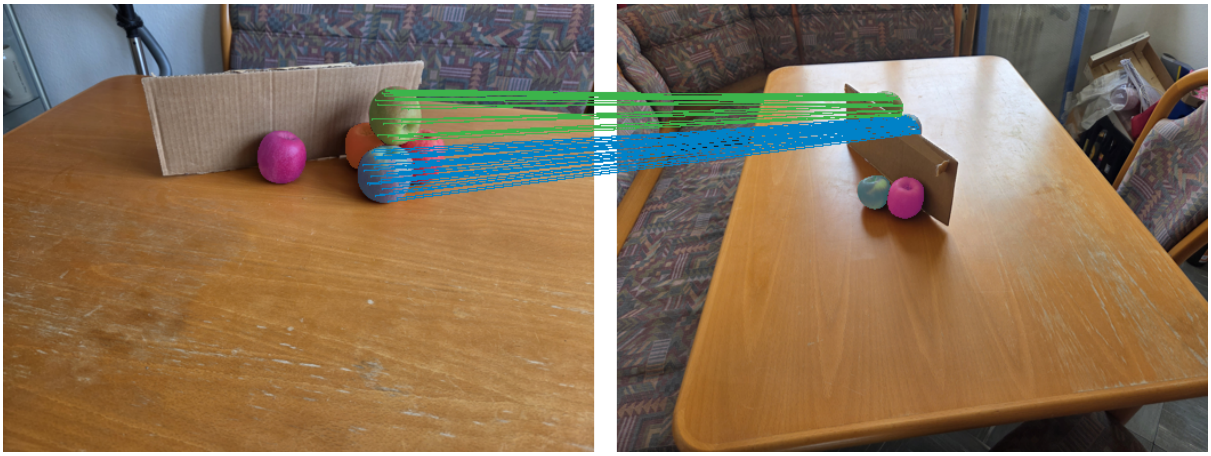


**Figure 4.3:** Sampled pixel correspondences from view 1 (left) to view 4 (right) of the running example, colored by instance. Apples hidden by the partition in one of the views receive hardly any correspondences.

with the full mask in the other view, whereas normalization based on the union of both masks would penalize the occlusion. Since multiple pixels of $a$ may correspond to the same pixel of $b$, the ratio can be greater than one, which is why we restrict it to one.

### 4.3.3 Pairwise Matching

For a pair of views $i$ and $j$ with $n_i$ and $n_j$ masks, the costs form a matrix $C \in \mathbb{R}^{n_i \times n_j}$, and we solve the rectangular assignment problem from Equation Equation (3.1) with masks of the two views taking the roles of the sets $A$ and $B$. No candidate is excluded in advance: all entries, including those with no overlap, are included in the optimization so that the solution is derived from complete information about the overlaps. Weak matches are excluded only afterward, thereby implementing the partial matching described in Section 3.3.3: An assigned pair $(a, b)$ becomes an edge only if

$$o_{ab} > \tau_{\text{ov}}. \tag{4.5}$$

Both thresholds operate at different levels. $\tau_{\text{corr}}$ determines whether a single pixel has a corresponding pixel, while $\tau_{\text{ov}}$ determines whether the mask as a whole matches. The few correspondences at the point where two adjacent apples touch usually result in an overlap that is smaller than $\tau_{\text{ov}}$. If this is not the case, the one-to-one assignment still prefers the true counterpart, as the following example shows.

Table 4.1 shows the overlaps between view 1 and view 4 of the example. Only two apples are visible in both views; the cardboard hides the remaining apples from view 1 in view 4 and the other way around. Thus, only two entries are close to one, and these pairs become edges between the two views. Since the rectangular assignment pairs each mask of the view with fewer masks with a partner, the two remaining masks of view 4 also receive partners in view 1, but with an overlap close to zero, which is why the threshold $\tau_{\text{ov}}$ removes them again. The table also shows why a one-to-one assignment is necessary: the mask $a_2$ not only overlaps with its actual corresponding mask $b_3$, but also overlaps by 0.21 with the mask $b_2$ of the neighboring apple. This value exceeds $\tau_{\text{ov}}$, so a threshold alone would connect $a_2$ and $b_2$ and falsely merge the two apples. The assignment prevents this, since $b_2$ is already the best partner for $a_3$.

**Table 4.1:** Overlap $o_{ab}$ between the masks $a$ of view 1 (rows) and the masks $b$ of view 4 (columns) of the running example. Bold: pairs accepted as edges; italic: pairs rejected by $\tau_{\text{ov}} = 0.2$.

| | $b_1$ | $b_2$ | $b_3$ | $b_4$ |
|---|---|---|---|---|
| $a_1$ | *0.00* | 0.05 | 0.00 | 0.00 |
| $a_2$ | 0.00 | 0.21 | **1.00** | 0.00 |
| $a_3$ | 0.00 | **1.00** | 0.00 | 0.00 |
| $a_4$ | 0.00 | 0.00 | 0.00 | 0.00 |
| $a_5$ | 0.00 | 0.00 | 0.00 | *0.01* |

### 4.3.4 Aggregation into Multi-View Instances

Confirmed matches form the edges of the graph from Section 3.2.1. The Union-Find structure is initialized with a set for each mask and `union` is called for each edge. After all image pairs have been processed, the sets form the connected components of the graph. Each component represents a hypothetical apple and is referred to below as an *instance*.

Because connectivity is transitive (Section 3.2.1), an apple observed in, for example, five views is recovered as a single component of five masks, even though no single matching step compared all five masks at once.

Unlike CCGS, which transfers labels from each individual image to the next, we match all $N(N-1)/2$ image pairs. In a sparse 360-degree capture, two images showing the same apples are not necessarily adjacent in any order, for example, two views taken at different heights on the same side of the tree. By matching all pairs, such views are directly connected to one another, rather than relying on a chain of intermediate views that breaks as soon as a connection fails.

The transitivity of the graph also has a drawback: a single incorrect edge connects two components, including all elements contained within them. Therefore, we add a constraint that is not expressed by the graph's formulation alone. An apple appears at most once in each view, so a component must not contain two masks of the same image. Before adding an edge, we check whether the components of its two nodes share the same view; if so, the edge is discarded. This excludes unions that contradict the segmentation of a single view. This constraint has two consequences. First, since the view pairs are processed in a specific order and each check depends on the edges added previously, the result may depend on this order. Second, if, for example, SAM3 splits an apple in a view into two masks because a branch obscures its central part, the constraint preserves both parts as separate components. Such fragments can be reunited through clustering.

Figure 4.4 shows the resulting graph for the running example. The 25 masks of the five views form seven connected components, one per apple. Since only 18 of the 32 accepted matches are required to connect these components, each component would stay connected even if individual matches failed, which is the advantage of matching all pairs of views.

## 4.4 Apple Model: Sphere Fitting and Filtering

The association returns a set of instances, each of which is a hypothesis about a single apple. Before counting them, we fit a geometric apple model to each instance. Since apples are nearly spherical, we

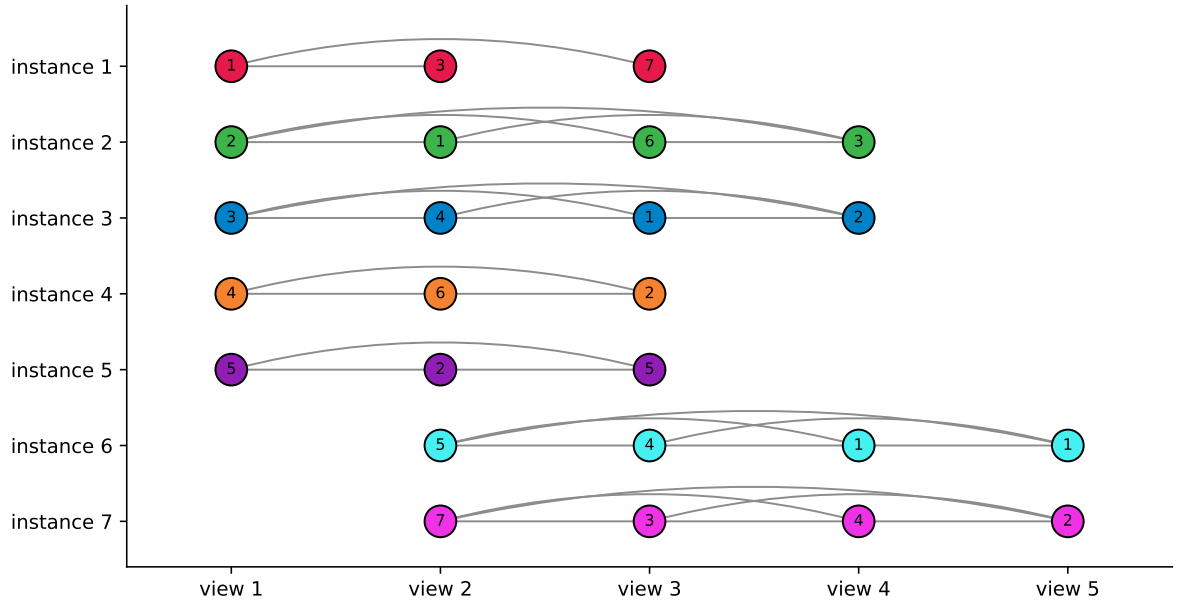



**Figure 4.4:** Mask graph of the running example. Each column corresponds to a view, each node to an instance mask (labeled with its in-view identifier), and each row to one connected component. The edges represent accepted matches.

use a sphere. The model has two purposes: it allows us to reject instances that cannot correspond to a single apple, and its center provides a compact representative of the instance for the clustering stage (Section 4.5).

For an instance $\mathcal{I}_k$, we aggregate all its mask points

$$\mathcal{P}_k = \{\, P_i(y) \,:\, (i,m) \in \mathcal{I}_k,\ y \in \mathcal{M}_{i,m} \,\}, \tag{4.6}$$

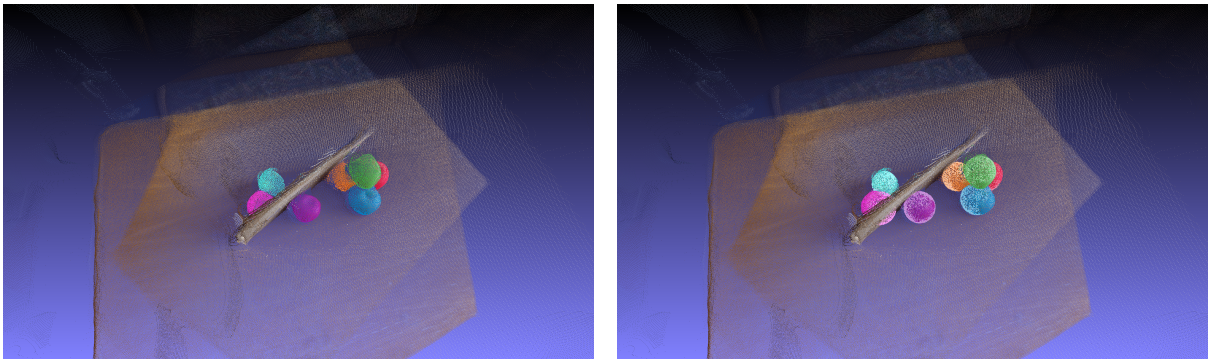
without considering invalid pixels. Each view captures only the side of an apple facing the camera. Thus, the points of an instance cover the part of the apple surface that is visible in at least one of its views, a larger part if the instance consists of masks from multiple views, and only a single cap if it consists of a single mask.

Before fitting, we discard instances supported by fewer than $m_{\min}$ masks. Given enough captured views, we can assume every apple is visible in many views, and so an instance that consists of a single mask is usually a fragment of an apple whose correspondences failed rather than an apple of its own. For sparse captures with a small number of views we set $m_{\min} = 1$, since more apples are likely to be seen in a single view only and should not be discarded for this reason.

We compute the sphere using RANSAC (Section 3.4). A minimal sample consists of four points that determine a unique sphere, provided they do not all lie in the same plane. A point $p$ belongs to the consensus set of a candidate sphere with center $c$ and radius $r$ if its distance to the sphere's surface does not exceed the tolerance $\delta$:

$$\big|\, \|p - c\|_2 - r \,\big| \le \delta. \tag{4.7}$$

After a fixed number of iterations, the sphere with the largest consensus set is chosen, yielding the center point $c_k$ and radius $r_k$ for the instance. We do not refit this sphere based on its consensus set.



(a) (b)

**Figure 4.5:** Sphere fitting in the running example. (a) The seven instances in the reconstructed scene, one color per instance. (b) Their fitted spheres, shown as points. All spheres satisfy the radius criterion.

The consensus criterion makes the fit robust against outliers that remain after filtering in Section 4.2.3. Instances with fewer than four points, for which no sphere can be determined, are discarded.

Using the center of the sphere instead of the centroid of the points as the representative of an instance is essential for the clustering. Since the points often cover only the visible side of the apple, their centroid is shifted toward the observing cameras. Thus, two fragments of the same apple observed from opposite sides would have centroids on opposite sides of the apple, while the centers of their fitted spheres are close to each other.

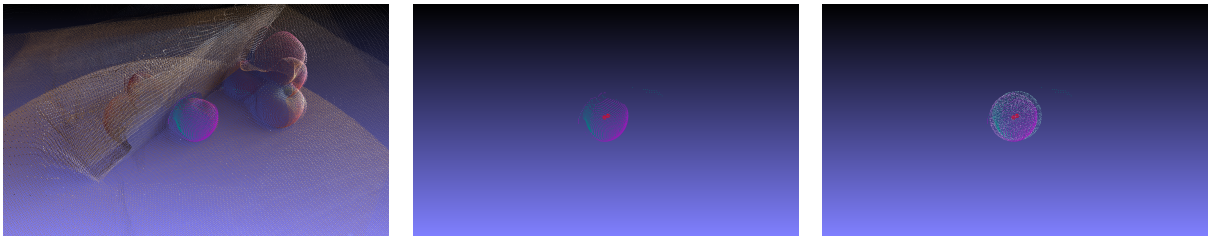
Finally, we verify the plausibility of the fitted radius. An instance is accepted only if

$$r_{\min} \leq r_k \leq r_{\max}, \tag{4.8}$$

and otherwise rejected. For instances that consist of a single mask, we use a looser upper bound, since the points of a single view cover only a cap of the apple and constrain its radius less well. A radius smaller than $r_{\min}$ usually results from small fragments or incorrect masks. A radius exceeding $r_{\max}$ indicates that the instance cannot be a single apple, for example because it contains points from multiple apples or points that do not belong to an apple at all, or because the points from a single view cover too small a portion of the surface to constrain the radius. In the base pipeline, such instances are discarded. The consequences for instances that include multiple apples are analyzed in Section 4.6. Figure 4.5 shows the fitted spheres for the running example.

## 4.5 Counting: Clustering of Sphere Centers

Since the association links masks exclusively based on correspondences between pairs of views, two instances can describe the same apple even if none of their masks have ever been matched, for example when the views observing them do not share any visible part of the apple's surface, or when SAM3 has split the apple into two masks within a single view. Such split errors result in multiple instances for a single apple, each of which has its own sphere. Since the centers of spheres representing fragments of the same apple are close to one another, these fragments can be merged in 3D space, regardless of the correspondences that failed during the association.



(a) (b) (c)

**Figure 4.6:** Clustering in the running example, with the correspondences between views 1 and 3 switched off. (a) The two instances of one apple in the reconstructed scene. (b) Their points with the centers of their fitted spheres (red). (c) The fitted spheres nearly coincide, so the clustering merges both instances.

Therefore, we group the centers $c_k$ of all accepted instances using agglomerative clustering with complete linkage (Section 3.5). Two clusters are merged only if all pairs of their centers are closer to each other than the threshold $d_{\text{cl}}$, so that the diameter of each cluster remains below $d_{\text{cl}}$. Each resulting cluster is considered a single apple, and the number of clusters corresponds to the final count.

The threshold is bounded on both sides. It must be greater than the distance between the centers of the spheres assigned to different fragments of the same apple, so that these fragments are merged. Simultaneously, it must be smaller than the distance between the centers of two adjacent apples, which is approximately the diameter of an apple when the apples are touching, so that neighboring apples are not merged into a single one. The complete linkage ensures that this limit applies to the entire cluster and not just to consecutive pairs of centers. Thus, a row of closely spaced apples cannot be merged into a single cluster, as would be possible when using DBSCAN (Section 3.5).

With the parameters used so far, the running example already recovers every apple as a single instance, so the clustering has nothing to merge. To illustrate the clustering in isolation, Figure 4.6 uses only views 1 and 3 and switches off the correspondences between them ($\tau_{\text{corr}} = 0$), so that every mask forms an instance of its own. The two masks of the same apple thus yield two instances that cover different parts of its surface. The centers of their spheres lie only 0.009 apart, about a fifth of an apple radius and far below $d_{\text{cl}} = 0.040$, so the clustering merges the two instances into one apple.

## 4.6 Error Analysis of the Base Pipeline

The individual steps of the base pipeline try to correct split and merge errors at different stages, though not to the same extent. This section analyzes where these errors occur and which of them the base pipeline can correct.

Split errors occur whenever the association process fails to combine masks of the same apple. Correspondences between two masks of one apple are missing if both views observe different parts of its surface, if the predictions of the same surface point in both views differ by more than $\tau_{\text{corr}}$, or if SAM3 splits the apple into multiple masks within a single view. Matching all pairs of views makes the aggregation robust against individual failed matches (Section 4.3.4), and clustering brings together fragments whose centers are close to each other (Section 4.5). Thus, a split error persists only when the centers of the fragments are farther apart than $d_{\text{cl}}$, which mainly occurs when a fragment consists of a small cap whose sphere is poorly constrained.

Merge errors arise from faulty edges. Correspondences between adjacent apples occur wherever their predicted surfaces come closer than $\tau_{\text{corr}}$: at their point of contact and, more generally, wherever the deviation of the reconstruction is comparable to the distance between the apples. The one-to-one assignment eliminates an incorrect edge as long as the true counterpart of a mask is visible in the other view. If an apple is occluded in view $j$, its mask in view $i$ has no true counterpart there and is assigned to the mask of a neighboring apple, provided that their overlap exceeds $\tau_{\text{ov}}$. Due to the transitivity of connectedness, such an edge connects the complete instances of both apples, and additional false edges can chain multiple apples into a single instance. The same-view constraint blocks such a union only if the two components already contain masks of a common view when the edge is processed.

In the base pipeline, a merged instance usually does not satisfy the radius criterion: its points belong to multiple apples, so the fitted sphere is larger than $r_{\text{max}}$ and the instance is discarded along with all apples it contains. If the merged apples are only partially observed, the fitted radius can remain plausible, and the instance is counted as a single apple. In any case, a merge error leads to undercounting. Unlike a split error, it cannot be corrected by the clustering, which only merges instances and never separates them.

Thus, both types of errors pull the thresholds in opposite directions. A large $\tau_{\text{corr}}$ and a small $\tau_{\text{ov}}$ connect the apple fragments even in the case of noisy correspondences, but they also cause incorrect edges between adjacent apples. Strict thresholds prevent false edges, but disrupt the connections within an apple. If the deviation between the predictions for the same surface point from different viewpoints is as large as the distance between neighboring apples, no single choice of thresholds avoids both errors. Furthermore, both errors can cancel each other out in the count, which is why a count close to the actual result does not necessarily indicate a correct association. This motivates the refinement described in the next section: instead of discarding an instance that fails the apple model, it is re-evaluated using stricter thresholds.

## 4.7 Iterative Refinement

In the refined method, the exclusion of implausible instances is replaced by a re-examination. It follows the base pipeline up to the apple model, but treats an instance that fails the model as a potential merge error and attempts to split it.

An instance is considered too large if its fitted radius exceeds $r_{\text{max}}$ or if the extent of its points exceeds the threshold $e_{\text{max}}$. The radius alone is not sufficient to reliably detect merged instances: since RANSAC selects the sphere with the largest consensus set, it may fit a plausible sphere to the points of a single apple and treat the points of the remaining apples as outliers. Therefore, we additionally measure the extent $e_k$ of an instance, defined as the greatest span of its points along the coordinate axes between the 2nd and 98th percentiles. For an instance whose points belong to multiple apples, the extent exceeds the diameter of a single apple, even if its fitted radius is plausible. As for the radius, instances that consist of a single mask receive a looser bound, since the points of a single view cover only a cap of the apple and are more strongly influenced by the smearing at the mask edge.

An instance that is too large and consists of more than one mask is split by repeating the association (Section 4.3) only for those masks, with stricter thresholds. In iteration $t$, the thresholds are given by

$$\tau_{\text{corr}}^{(t)} = s^t \, \tau_{\text{corr}}, \qquad \tau_{\text{ov}}^{(t)} = \tau_{\text{ov}} + t \, \Delta, \tag{4.9}$$

with a shrink factor $0 < s < 1$ and an increment $\Delta > 0$. A stricter correspondence threshold excludes matches between neighboring apples whose predicted surfaces are farther apart than two predictions of the same surface, while a higher overlap threshold additionally rejects weak matches. If the assignment produces multiple components, each of them is compared to the apple model again in the next iteration, and parts that pass the test are accepted, while parts that are still too large are split further. The process ends after at most $K$ iterations.

If a stricter association does not split the instance, or if the maximum number of iterations is reached, the instance is accepted even if it does not match the model. In contrast to the base pipeline, the refined pipeline thus never discards an instance with multiple masks because it is too large, since otherwise every apple it contains would be lost. Single-mask instances that are too large cannot be split and are discarded, as are instances whose radius is smaller than $r_{\min}$. Finally, all instances accepted during the iterations are clustered as before. A split often separates a few masks of an apple from an instance that otherwise belongs to a neighboring apple, and these masks form a fragment that the clustering merges with the remaining masks of their apple.

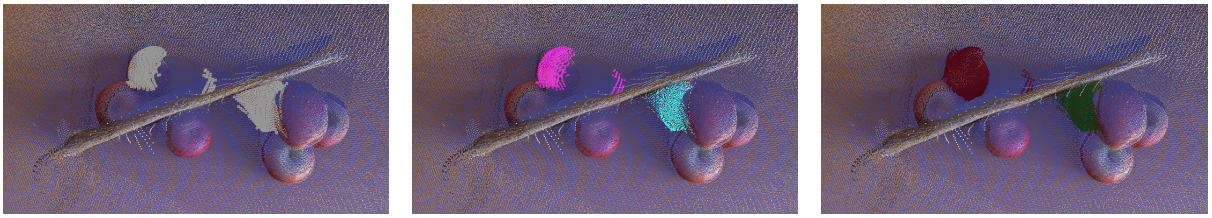
During splitting, only the masks of the instance itself are considered. Since the masks of other apples are not involved in this association, the one-to-one assignment cannot fall back on the true counterpart of a mask. A mask that was attached to the wrong apple therefore stays with it as long as its overlap with that apple exceeds the stricter threshold.

Figure 4.7 illustrates the refinement on the running example. To induce merge errors, we use deliberately permissive thresholds of $\tau_{\text{corr}} = 0.1$ and $\tau_{\text{ov}} = 0.05$. The original graph again consists of seven components, three of which contain masks of two different apples. Two of these instances exceed $r_{\max}$; the base pipeline discards them and thereby loses two apples completely, resulting in a count of five. The refined pipeline, with $s = 0.25$ and $\Delta = 0.1$, re-associates both instances with $\tau^{(1)}_{\text{corr}} = 0.025$ and $\tau^{(1)}_{\text{ov}} = 0.15$ and splits each of them into two instances, each containing one apple. Subsequently, the clustering process combines the split-off masks with the remaining masks of their respective apples, thereby finding all seven apples. The third mixed instance, consisting of three masks of one apple and one mask of another, satisfies the radius criterion and violates only the extent criterion; however, a stricter association does not split it, so it is accepted as it is. Since the remaining masks of the second apple form an instance of their own, the count remains correct, and five of the seven apples are represented by exactly one instance that contains no other apple, compared to three in the base pipeline.

## 4.8 Implementation Details

The pipeline is implemented in Python. For the reconstruction, we use the reference PyTorch implementation of VGGT with the publicly released VGGT-1B model [40] and its default preprocessing, which resizes the images to a width of 518 pixels while preserving their aspect ratio. All reconstructions are computed on a single NVIDIA A100 GPU with 40 GB of memory, while the remaining pipeline is run on an Intel Core i5-13600KF CPU. The instance masks are obtained using SAM3 [4] with the text prompt “apple” on an NVIDIA GeForce RTX 3070.

For each pair of views, the nearest-neighbor search of Equation Equation (4.1) uses a k-d tree built over the valid apple points of view $j$ [39]. The intersections of Equation Equation (4.3) for all



(a) (b) (c)

**Figure 4.7:** Iterative refinement in the running example, with deliberately permissive thresholds. (a) An instance of the initial graph (gray) that contains masks of two apples on both sides of the partition. (b) The stricter association splits it into two instances with one apple each. (c) After clustering, the split-off part of the left apple is merged with the remaining masks of that apple.

**Table 4.2:** Parameters that are the same for all datasets.

| Parameter | Value |
|---|---|
| SAM3 confidence threshold | 0.5 |
| Minimum mask area | 100 pixels |
| Neighbors in the outlier filter, $k_f$ | 12 |
| Overlap threshold, $\tau_{ov}$ | 0.2 |
| RANSAC iterations | 500 |
| Shrink factor, $s$ | 0.25 |
| Overlap increment, $\Delta$ | 0.1 |
| Maximum refinement iterations, $K$ | 4 |
| Random seed | 42 |

pairs of masks are then obtained by counting the correspondences per pair of instance identifiers. The assignment problem (Section 3.3) is solved with the implementation in SciPy [39], which uses one of the faster variants [7]. The view pairs are processed in lexicographic order. The outlier removal uses Open3D [45], the sphere fitting pyRANSAC-3D [28], and the clustering the agglomerative clustering of scikit-learn [34] with complete linkage. A fixed random seed ensures the reproducibility of results.

The apple radius $\hat{r}$ used to derive the scene-dependent thresholds (Section 5.1.2) is estimated from the point clouds of individual masks. For each mask, we compute the extent of its points along their second principal axis, which remains largely unaffected by smearing along the viewing direction, and set $\hat{r}$ to half the upper quartile of these extents. The parameters that are consistent across all datasets are listed in Table 4.2.

# Chapter 5

# Results

We evaluate the counting pipeline using three datasets with increasing levels of complexity: a table scene with a few apples, a synthetic tree with full ground truth, and a real apple tree. For each dataset, we first provide the count and then examine how it was determined: how well the association groups the masks of each apple, how reliable the correspondences between views are, and, where the ground truth allows it, how accurate the reconstruction is. This is necessary because split and merge errors can cancel each other out, which is why a count that is close to the ground truth does not by itself indicate a correct association (Section 4.6).

Following the description of the datasets and parameters (Section 5.1), we present the results for the table scene (Section 5.2), the synthetic tree scene (Section 5.3), and the real tree (Section 5.4). The chapter concludes with a comparison between the datasets, the method's runtime and memory requirements, and the limitations of the evaluation (Section 5.5).

## 5.1 Experimental Setup

### 5.1.1 Datasets

The *table scene* is the running example of Chapter 4: seven apples on a table, captured from five viewpoints using a smartphone and segmented with SAM3. Since no ground truth exists for its association, the manually checked association of a pipeline run serves as the reference.

The *synthetic tree* is a scene built in Blender [2] to obtain full ground truth. It consists of a tree model with 38 apples, set against a neutral background and captured by 48 cameras arranged in three rings of 16 cameras at different heights. For every view, we additionally render the identifier of the visible apple and the world-space position of its surface at every pixel. From these, we obtain instance masks with cross-view consistent apple identities, the true 3D position of every apple pixel, and the center and radius of every apple. We use these masks instead of SAM3, which misses the apples in about half of the rendered views, so that the evaluation isolates the reconstruction and association analysis from segmentation errors.

The real tree, *tree_02*, is taken from the FruitNeRF dataset [29], which provides 366 images with a size of $6000 \times 4000$ pixels. We select 64 views evenly distributed over 360 degrees and segment them with SAM3, which detects apples in all but two of them. The only ground truth is the manually counted

**Table 5.1:** Datasets. Size: median mask area at the VGGT input resolution, in $14 \times 14$ pixel patches.

| Dataset | Views | Apples | Masks | Ground truth | Apples/image | Size |
|---|---|---|---|---|---|---|
| Table scene | 5 | 7 | SAM3 | reference | 5.0 | 3.8 |
| Synthetic tree | 48 | 38 | Blender | masks, geometry | 18.8 | 0.29 |
| tree_02 | 64 | 113 | SAM3 | count | 33.2 | 0.47 |

(a) (b)

**Figure 5.1:** Example views of the two trees. (a) Synthetic tree, rendered in Blender. (b) tree_02 from the FruitNeRF dataset [29]. In both, each apple covers only a small part of the image.

number of apples, 113 [29].

Table 5.1 shows how the complexity increases from the table scene to the trees. The trees contain four to seven times as many apples per image, and their apples appear so small that, at the input resolution of VGGT, they cover less than half of one of the $14 \times 14$ pixel patches from which VGGT forms its tokens (Section 3.1.3), compared to almost four on the table. Figure 5.1 shows an example view of each tree.

### 5.1.2 Parameters

Parameters which are the same for all datasets are listed in Section 4.8. The remaining thresholds, which depend on the scale of the reconstruction, are given in Table 5.2. For the table scene and tree_02, we derive them with fixed ratios from an estimate $\hat{r}$ of the apple radius in the units of the reconstruction, which we obtain from the size of the point clouds of individual masks. The thresholds of the synthetic tree were set during the development of the method on this scene, before its evaluation against the ground truth. We set $m_{\text{min}} = 1$ for the table scene, which has only five views, and $m_{\text{min}} = 2$ for both trees.

## 5.2 Table Scene

The pipeline counts all seven apples of the table scene. To verify that the counted apples match the actual ones, we compare each final cluster with each apple using the intersection over union (IoU),

**Table 5.2:** Scene-dependent parameters, in the units of the respective reconstruction. Rule: ratio to the estimated apple radius $\hat{r}$ used for the table scene and tree_02. For $r_{\max}$ and $e_{\max}$, the second value applies to instances consisting of a single mask.

| Parameter | Rule | Table scene | Synthetic tree | tree_02 |
|---|---|---|---|---|
| $\hat{r}$ | – | 0.040 | – | 0.013 |
| $\tau_{\text{corr}}$ | $0.5\,\hat{r}$ | 0.020 | 0.020 | 0.0065 |
| $r_{\text{f}}$ | $0.35\,\hat{r}$ | 0.014 | 0.0076 | 0.0046 |
| $\delta$ | $0.25\,\hat{r}$ | 0.010 | 0.008 | 0.0033 |
| $r_{\min}$ | $0.5\,\hat{r}$ | 0.020 | 0.008 | 0.0065 |
| $r_{\max}$ | $1.5\,\hat{r}$ / $2\,\hat{r}$ | 0.060 / 0.081 | 0.050 / 0.070 | 0.020 / 0.026 |
| $e_{\max}$ | $2.6\,\hat{r}$ / $3\,\hat{r}$ | 0.105 / 0.121 | 0.090 / 0.100 | 0.034 / 0.039 |
| $d_{\text{cl}}$ | $\hat{r}$ | 0.040 | 0.014 | 0.013 |
| $m_{\min}$ | – | 1 | 2 | 2 |

which is calculated based on the mask pixels across all views, and we consider a pair with an IoU above 0.5 as a true positive result. Since neither the masks of different apples nor the masks of different clusters overlap, a cluster can exceed this threshold with at most one apple, so the true positives are unique. From them, we compute the precision (true positives per apple counted), the recall (true positives per true apple), and their harmonic mean, the *F1 score*. On the table scene, each cluster corresponds to exactly one apple, and the F1 score is 1.0.

The association already achieves this before the clustering: the 25 masks of the five views form exactly seven components in the initial graph (Figure 4.4), one for each apple, and none of them contains masks from other apples. To compare this between datasets, we use the number of initial components per apple, which is 1 for an ideal association, larger when the masks of an apple are split over several components and smaller when components merge several apples. On the table scene, it is exactly 1, and all seven instances also pass the radius criterion of the apple model (Figure 4.5).

This result is based on reliable correspondences between the views. For each pair of views and each apple pixel, we determine the nearest apple point in the other view as in Equation Equation (4.1), and use the reference identities to check whether it belongs to the same apple. The *correspondence precision* is the proportion of accepted correspondences that connect the same apple, and the *correspondence recall* is the proportion of apple pixels that receive a correct correspondence, among those whose apple is visible in the other view. Low precision leads to merge errors, while low recall leads to split errors. At the threshold $\tau_{\text{corr}}$ used by the pipeline, the table scene achieves a correspondence precision of 0.96 and a recall of 0.80, meaning almost every accepted correspondence is correct, and most of the possible ones are found. Since each view depicts only one side of an apple, the recall cannot reach 1 even with a perfect reconstruction.

The result is also robust to the choice of thresholds: it remains unchanged when $\tau_{\text{corr}}$ is raised from 0.02 to 0.1. Merge errors occur only when the overlap threshold is lowered at the same time, to the deliberately permissive values of Section 4.7 ($\tau_{\text{corr}} = 0.1$, $\tau_{\text{ov}} = 0.05$). Without the refinement, the pipeline then discards the merged instances and counts five apples (F1 score 0.50). With the refinement, it splits them and again counts all seven (F1 score 1.0).

(a) (b)

**Figure 5.2:** Groups of masks on the synthetic tree, shown by their apple points and colored by their correctness (green: exactly one apple, not split; orange: one apple, split over several groups; red: several apples; gray: discarded). (a) Components of the initial graph, in which only a few apples are recovered correctly. (b) Final clusters of the refined pipeline: more apples are recovered correctly, but split and merge errors remain.

## 5.3 Synthetic Tree

On the synthetic tree, the pipeline counts 43 apples, five more than the 38 actual ones. However, only 29 of the counted apples correspond to an actual apple, so that 9 of the 38 apples are not found and 14 counted apples are incorrect. The resulting F1 score of 0.72 shows that the count is closer to the ground truth than the association behind it.

Unlike on the table scene, the association does not keep the masks of an apple together: the initial graph consists of 95 components, 2.5 per apple. To locate these errors, we use the ground truth identifiers. The *split excess* counts, summed over all apples, the groups containing masks of an apple beyond the first; the *merge loss* counts, summed over all groups, the apples in a group beyond the first. In the initial graph, the split excess is 114 and the merge loss 57, so both types of error occur in large numbers. The refinement and the clustering reduce them to 39 and 34, but these remaining errors largely cancel each other out in the count: $38 + 39 - 34 = 43$. Figure 5.2 shows where the errors occur in the reconstructed scene.

The errors originate in the correspondences between the views. At the threshold used by the pipeline, only 82 % of the accepted correspondences connect the same apple, compared to 96 % on the table scene, and only 55 % of the possible correspondences are found, compared with 80 %. Thus, nearly one in five accepted correspondences links two different apples, leading to merge errors, while nearly half of the possible ones are missing, which causes split errors. Changing the threshold does not resolve this (Figure 5.3): a stricter threshold increases precision only slightly, while recall decreases sharply, and a looser one does the opposite. Unlike on the table scene, no single threshold guarantees both high precision and high recall.

The ground truth of the synthetic tree allows us to link these errors to the reconstruction. Figure 5.4 shows the apple points within the reconstructed scene, colored by their actual apple: the points of each apple spread well beyond its size, and the points of neighboring apples overlap. To quantify this, we align the predicted apple points with their actual positions using a similarity transformation [37] and

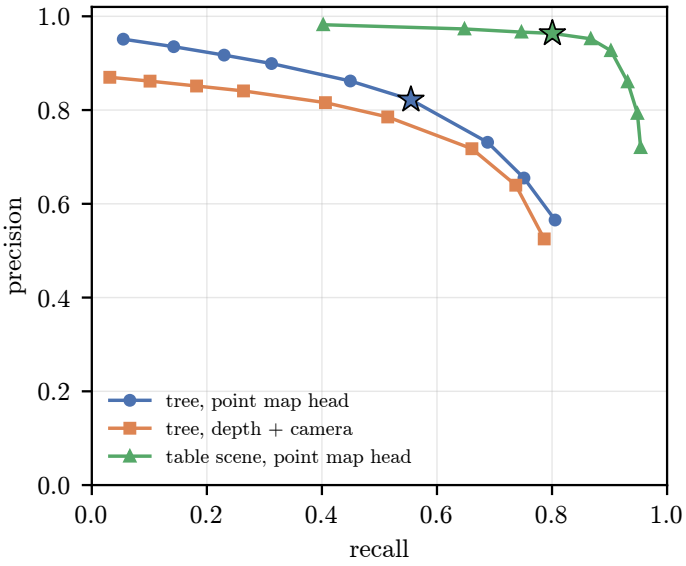


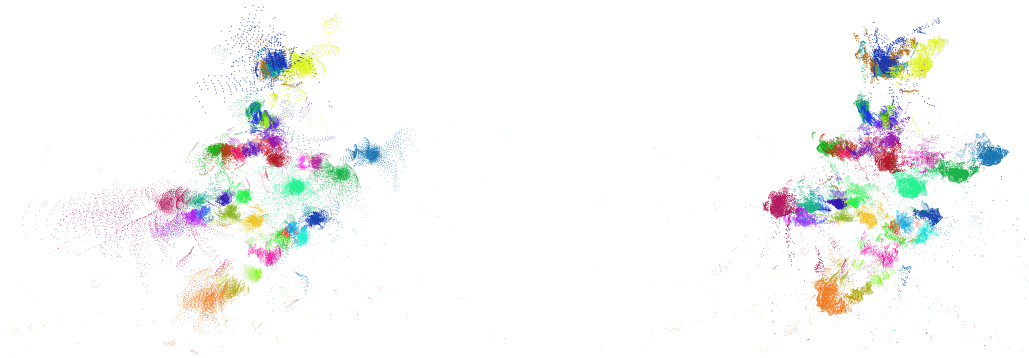

**Figure 5.3:** Correspondence precision against recall, with the distance threshold $\tau$ as curve parameter, for the synthetic tree (point map head and depth with camera) and the table scene. Stars: threshold used by the pipeline.

subtract the median offset of each apple, so that only the scatter around the individual apple remains. The distance of a point from its actual position, divided by the radius of the respective apple, then has a median of 1.21. This means that a typical point lies more than an apple radius away from where it belongs.

The position of an apple as a whole is more accurate than that of its individual points. A sphere with the true radius, fitted to all points of the apple using the true masks, is located at a distance of 0.52 radii from the true center in the median. This is the best position the pipeline could calculate with a perfect association, bounded by the reconstruction quality. For the apples that are counted correctly, the center of the cluster, the mean of the sphere centers of its instances, lies 1.04 radii from the true center, twice as far.

All of these errors are larger for the points unprojected from the depth maps with the predicted cameras (Figure 5.4b): their median distance is 1.59 radii, the center of their fitted spheres is 0.69 radii away from the actual center, and their correspondence precision is lower at every recall (Figure 5.3). This confirms the choice of the point map head in Section 4.2.

Finally, we examine whether the parameters of the pipeline can compensate for these errors (Table 5.3). Since tuning the parameters on the only scene with ground truth would bias the evaluation, these variations only show how sensitive the result is; all results above use the parameters fixed beforehand. Without the refinement, the pipeline counts only 28 apples, but with fewer incorrect ones; the refinement recovers more apples at the cost of more incorrect ones and raises the F1 score slightly, from 0.70 to 0.72. The outlier filter has the largest effect: without it, the F1 score drops to 0.61, while a stricter filter raises it to 0.79, since removing more of the scattered points yields cleaner spheres and fewer merged instances. A confidence filter that removes the same share of points reaches a similar F1 score, but locates the apples less accurately (1.50 instead of 1.04 radii). More views make the



(a) (b)

**Figure 5.4:** Apple points of the synthetic tree before outlier removal, one color per ground-truth apple, within the reconstructed scene. (a) Point map head: most points of each apple lie in a dense core, surrounded by sparse, isolated outliers of the kind removed by the outlier filter (Section 4.2.3). (b) Depth maps unprojected with the predicted cameras: fewer isolated outliers, but the points of each apple scatter more widely.

**Table 5.3:** Sensitivity of the count on the synthetic tree (38 apples). Correct: counted apples that correspond to an actual apple (IoU above 0.5). Default: parameters of Table 5.2 with refinement, radius filter with $k_f = 12$, and all 48 views.

| | Variant | Count | Correct | F1 |
|---|---|---|---|---|
| Refinement | without | 28 | 23 | 0.70 |
| | with (default) | 43 | 29 | 0.72 |
| Outlier filter | none | 41 | 24 | 0.61 |
| | radius, $k_f = 12$ (default) | 43 | 29 | 0.72 |
| | radius, $k_f = 30$ | 43 | 32 | 0.79 |
| | confidence | 38 | 27 | 0.71 |
| Views | 12 | 32 | 26 | 0.74 |
| | 24 | 39 | 29 | 0.75 |
| | 36 | 42 | 27 | 0.68 |
| | 48 (default) | 43 | 29 | 0.72 |

reconstruction more accurate: on the views common to all subsets, the scatter of the points decreases from 1.50 radii with 12 views to 1.21 with 48. However, the F1 score does not follow, since every additional view adds pairs of views and with them opportunities for incorrect edges. The parameters thus shift the balance between split and merge errors or reduce them to some extent, but none of them removes them.

## 5.4 Real Tree

On tree_02, the association largely fails. The 2055 masks of the 62 views with detections form 1778 components in the initial graph, 15.7 per apple, compared to 2.5 on the synthetic tree and 1 on the table scene. Almost no masks are matched to masks of other views, so most masks remain components of their

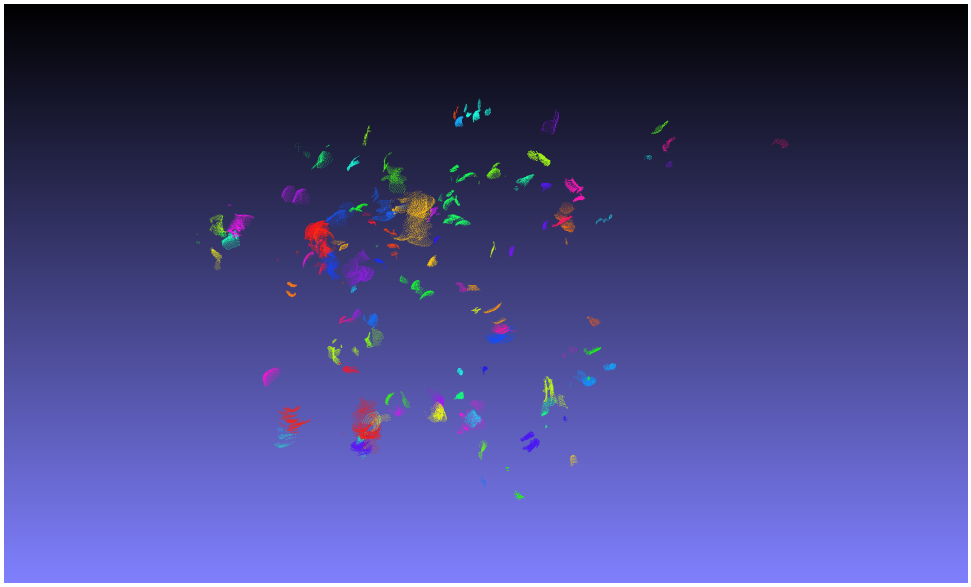


**Figure 5.5:** Final clusters on tree_02 with all thresholds scaled by a factor of 2, one color per cluster.

own, which are discarded with $m_{\text{min}} = 2$. As a result, the pipeline counts only 38 apples, about a third of the 113 counted manually. Since the components are so small, the refinement has hardly any instances to split and changes the count by only one apple. FruitNeRF, which optimizes a reconstruction of the same tree from about 350 images, counts between 86 and 114 apples, depending on the segmentation masks and the model size [29].

Adjusting the thresholds does not solve the issue. Multiplying all thresholds in Table 5.2 by 1.5 and 2 raises the count to 73 and 98, but even then more than a hundred instances are rejected because their fitted spheres are too large. Furthermore, the resulting clusters do not correspond to individual apples (Figure 5.5). In addition to many thin fragments of single surfaces, some clusters extend over several apple diameters, and different clusters overlap in the same regions. Relaxing the thresholds further until the count approaches 113 would therefore not make it more reliable, and we do not expect any configuration of the parameters to yield a reliable count for tree_02.

Without ground-truth geometry, the reconstruction of tree_02 can only be assessed indirectly. However, all observations point to the same cause. The segmentation is not the limiting factor: SAM3 detects 33 apples per image on average, but the masks of the same apple in different views are almost never matched, meaning that their predicted points do not come close to each other. The point clouds of individual masks are elongated, 1.6 times longer along the viewing direction than across it. As on the synthetic tree, where the ground truth shows that the points of an apple scatter by more than its radius, the apples are smaller than a single patch of VGGT, and the real capture adds wind, changing illumination, and segmentation errors.

## 5.5 Summary

Table 5.4 summarizes the results on the three datasets. As we move from the table scene to the trees, the apples appear smaller in the images, the correspondences between the views become less reliable, the

**Table 5.4:** Overview of the results. Size: median apple size at the VGGT input resolution, in $14 \times 14$ pixel patches. Correspondences: precision and recall at the threshold used by the pipeline. Components: components of the initial graph per apple.

| Dataset | Size | Correspondences | Components | F1 | Count / GT |
|---|---|---|---|---|---|
| Table scene | 3.8 | 0.96 / 0.80 | 1.0 | 1.00 | 7 / 7 |
| Synthetic tree | 0.29 | 0.82 / 0.55 | 2.5 | 0.72 | 43 / 38 |
| tree_02 | 0.47 | – | 15.7 | – | 38 / 113 |

association splits the apples into more components and the count departs further from the ground truth. Since the same pipeline counts every apple correctly in the table scene, where the reconstruction is most reliable, these errors originate in the reconstruction rather than the association itself. On the synthetic tree, the points of an apple scatter by more than its radius, and no parameter setting compensates for this.

**Runtime and memory.** All reconstructions were computed on a single NVIDIA A100 GPU with 40 GB of memory. VGGT's forward pass takes 3.3 seconds for the 48 views of the synthetic tree and 4.5 seconds for the 64 views of tree_02, excluding the loading of the model and the images. Its peak memory, including the memory reserved by the PyTorch allocator, amounts to 21.3 and 24.5 GB and increases linearly with the number of views, growing by approximately 0.2 GB per view, so that about 150 views could fit into the available memory at this resolution. The subsequent pipeline takes 19.7 and 17.4 seconds, so that a reconstruction and counting require about half a minute of computation. The most time-consuming step is the segmentation: SAM3 processes the 64 full-resolution images of tree_02 in about three minutes on an NVIDIA GeForce RTX 3070, so that a complete count takes about three and a half minutes. In comparison, FruitNeRF requires about 12 minutes of training per tree for its standard model and around 2.5 hours for its larger variant on an NVIDIA RTX A5000, in addition to the segmentation masks and the estimation of the camera poses with COLMAP [29].

**Limitations.** The synthetic tree is a single, static scene, and its ground-truth masks exclude the segmentation errors that add to the errors on real data. Its thresholds were set during the development of the method on this scene. The thresholds of all scenes are given in the units of each respective reconstruction and must be derived anew for each scene. Finally, only one real tree was evaluated, and only by its total count, since no ground truth per apple is available.

# Chapter 6

# Conclusions

This thesis investigated whether the feed-forward multi-view stereo model VGGT can replace the per-scene optimization of methods such as FruitNeRF for counting the apples on a tree. To do this, we developed a counting pipeline that associates the instance masks of SAM3 across multiple views through the point maps predicted by VGGT, following the principle of CCGS, aggregates the pairwise matches into multi-view instances, checks each instance against a sphere as an apple model, and clusters the sphere centers to obtain the count. An iterative refinement re-examines instances that fail the apple model with stricter thresholds instead of discarding them. We evaluated the pipeline using three datasets of increasing complexity: a table scene, a synthetic tree with full ground truth, and a real apple tree.

The answer to the central question of this thesis is twofold. In terms of runtime, VGGT delivers on its promise: reconstruction and counting take about half a minute, and even including the segmentation, a complete count takes only a few minutes, compared to minutes or hours of training for FruitNeRF. In terms of accuracy, however, its geometry is insufficient for complex scenes. On the table scene, where the apples appear large in the images, the correspondences between the views are reliable, and each apple is correctly counted. However, when it comes to trees, the apple points are distributed more widely than their radius, the matches become unreliable, and the association both splits and merges apples. On the synthetic tree, a count close to the ground truth results only from errors that cancel each other out, and on the real tree, the association fails almost completely. Since the same pipeline works on the table scene and no set of its parameters compensates for the errors on the trees, the limiting factor is the accuracy of the reconstruction at the scale of individual apples and not the association itself. The iterative refinement repairs merge errors when the geometry is accurate, as on the table scene, but on the trees it only shifts the balance between split and merge errors. There, the count alone would even be misleading, since split and merge errors largely cancel each other out, which underlines the need to evaluate the association behind a count.

We attribute this to the size of the apples compared to the resolution at which VGGT operates. To capture a whole tree, each image must cover the entire crown, so that at the input resolution of VGGT, an apple covers less than half of one of the patches from which the network forms its tokens, compared with almost four on the table. An apple that is smaller than a patch cannot be represented as an object of its own, and its points are interpolated from tokens that also describe leaves, branches and neighboring apples. FruitNeRF, which counts the apples of the same tree far more accurately, is

optimized on about 350 images at almost three times this resolution in each dimension. This is consistent with our explanation, although the two methods differ in more than the input resolution.

## 6.1 Future Work

The most direct way to improve the method is to make the apples larger relative to the patches of VGGT. This could be achieved by capturing the tree from a closer distance, reconstructing overlapping parts of the tree separately instead of the whole crown at once, or increasing the resolution of the input data. Each of these options increases the number of images or tokens per tree, which approaches with lower memory requirements, such as VGGT-Long [8], could make feasible. The synthetic scene makes it possible to test such variations against full ground truth and thus confirm or refute the explanation presented above. Furthermore, the same rendering pipeline can generate trees with exact geometry in any number, which could serve as training data to fine-tune VGGT on orchard scenes.

A second direction concerns the correspondences themselves. In our method, these are derived exclusively from the distances between predicted 3D points, which is precisely where the reconstruction is most inaccurate. VGGT-Segmentor [13] shows that the internal features of VGGT remain aligned at the level of objects even where its pixel-level correspondences drift. Associating the apple masks through such features rather than through individual points could therefore provide a more reliable basis on the trees, either on its own or in combination with the geometric distance.

Third, VGGT could serve as a starting point rather than a final reconstruction. Its point maps and camera parameters could be used to initialize a per-scene representation such as 3D Gaussian splatting [21], as used for fruit counting by FruitGaussian [18] and CountingFruit [24], which would then require only a short optimization. Such a hybrid approach could combine much of the speed of a feed-forward reconstruction with the accuracy of optimizing each individual scene.

# List of Figures


3.1 Overview of the VGGT architecture, taken from [40]. The input images are split into patch tokens by DINO and augmented with camera tokens. $L = 24$ blocks of frame-wise and global self-attention process the tokens of all images jointly, after which the camera head predicts the camera parameters and the DPT head the dense outputs. . . . . 8

3.2 A graph with three connected components (dashed). The nodes $u$ and $w$ are not adjacent, but they belong to the same component because they are connected by a path via $v$. . . 10

3.3 RANSAC fitting a circle to 2D points with outliers. Three points (minimal sample) determine a candidate circle, and the points within the tolerance band form its consensus set. The least-squares fit to all points (dashed) is pulled toward the outliers. . . . . 14

3.4 DBSCAN and agglomerative clustering with complete linkage on three compact groups, two of which are connected by a chain of points. DBSCAN links these two groups into a single cluster (left; gray: noise). Complete linkage keeps all three groups apart, and the chain forms a small cluster of its own (right). . . . . 15

4.1 Overview of the counting pipeline. VGGT predicts a point map for every view, and SAM3 segments the apples as instance masks. After outlier removal, the masks are matched between all pairs of views and aggregated into multi-view instances, which are checked against a sphere as apple model. In the refined pipeline, instances that are too large are re-associated with stricter thresholds (dashed). The centers of the accepted spheres are clustered to obtain the count. . . . . 18

4.2 Running example used throughout this chapter: five views of seven apples on a table, partially separated by a cardboard partition. . . . . 18

4.3 Sampled pixel correspondences from view 1 (left) to view 4 (right) of the running example, colored by instance. Apples hidden by the partition in one of the views receive hardly any correspondences. . . . . 21

4.4 Mask graph of the running example. Each column corresponds to a view, each node to an instance mask (labeled with its in-view identifier), and each row to one connected component. The edges represent accepted matches. . . . . 23

4.5 Sphere fitting in the running example. (a) The seven instances in the reconstructed scene, one color per instance. (b) Their fitted spheres, shown as points. All spheres satisfy the radius criterion. . . . . 24

4.6 Clustering in the running example, with the correspondences between views 1 and 3 switched off. (a) The two instances of one apple in the reconstructed scene. (b) Their points with the centers of their fitted spheres (red). (c) The fitted spheres nearly coincide, so the clustering merges both instances. . . . . 25
4.7 Iterative refinement in the running example, with deliberately permissive thresholds. (a) An instance of the initial graph (gray) that contains masks of two apples on both sides of the partition. (b) The stricter association splits it into two instances with one apple each. (c) After clustering, the split-off part of the left apple is merged with the remaining masks of that apple. . . . . 28

5.1 Example views of the two trees. (a) Synthetic tree, rendered in Blender. (b) tree_02 from the FruitNeRF dataset [29]. In both, each apple covers only a small part of the image. . 30
5.2 Groups of masks on the synthetic tree, shown by their apple points and colored by their correctness (green: exactly one apple, not split; orange: one apple, split over several groups; red: several apples; gray: discarded). (a) Components of the initial graph, in which only a few apples are recovered correctly. (b) Final clusters of the refined pipeline: more apples are recovered correctly, but split and merge errors remain. . . . . 32
5.3 Correspondence precision against recall, with the distance threshold $\tau$ as curve parameter, for the synthetic tree (point map head and depth with camera) and the table scene. Stars: threshold used by the pipeline. . . . . 33
5.4 Apple points of the synthetic tree before outlier removal, one color per ground-truth apple, within the reconstructed scene. (a) Point map head: most points of each apple lie in a dense core, surrounded by sparse, isolated outliers of the kind removed by the outlier filter (Section 4.2.3). (b) Depth maps unprojected with the predicted cameras: fewer isolated outliers, but the points of each apple scatter more widely. . . . . 34
5.5 Final clusters on tree_02 with all thresholds scaled by a factor of 2, one color per cluster. 35

# List of Tables


4.1 Overlap $o_{ab}$ between the masks $a$ of view 1 (rows) and the masks $b$ of view 4 (columns) of the running example. Bold: pairs accepted as edges; italic: pairs rejected by $\tau_{\text{ov}} = 0.2$. 22
4.2 Parameters that are the same for all datasets. . . . . 28

5.1 Datasets. Size: median mask area at the VGGT input resolution, in $14 \times 14$ pixel patches. 30
5.2 Scene-dependent parameters, in the units of the respective reconstruction. Rule: ratio to the estimated apple radius $\hat{r}$ used for the table scene and tree_02. For $r_{\max}$ and $e_{\max}$, the second value applies to instances consisting of a single mask. . . . . 31
5.3 Sensitivity of the count on the synthetic tree (38 apples). Correct: counted apples that correspond to an actual apple (IoU above 0.5). Default: parameters of Table 5.2 with refinement, radius filter with $k_{\text{f}} = 12$, and all 48 views. . . . . 34
5.4 Overview of the results. Size: median apple size at the VGGT input resolution, in $14 \times 14$ pixel patches. Correspondences: precision and recall at the threshold used by the pipeline. Components: components of the initial graph per apple. . . . . 36

# Bibliography


[1] Juan Ballena-Ruiz, Juan Arcila-Diaz, and Victor Tuesta-Monteza. "Automated Detection and Counting of Gossypium barbadense Fruits in Peruvian Crops Using Convolutional Neural Networks". In: *AgriEngineering* 7.5 (2025), p. 152. ISSN: 2624-7402. DOI: 10.3390/agriengineering7050152.

[2] Blender Online Community. *Blender 5.2 LTS – a 3D Modelling and Rendering Package*. https://www.blender.org. Amsterdam: Blender Foundation, 2026.

[3] Rainer Burkard, Mauro Dell'Amico, and Silvano Martello. *Assignment Problems*. Philadelphia, PA: Society for Industrial and Applied Mathematics (SIAM), 2009. ISBN: 978-0-898716-63-4. DOI: 10.1137/1.9780898717754.

[4] Nicolas Carion et al. *SAM 3: Segment Anything with Concepts*. 2026. arXiv: 2511.16719 [cs.CV]. URL: https://arxiv.org/abs/2511.16719.

[5] G. Ramesh Chandra, K. Sri Rama Murthy, S. Archana, B. Snehitha, G. Nishanth, M. Navya, and R. Sai Charan. "Fruit Detection and Counting Using Deep Learning". In: *2025 Eighth International Conference on Image Information Processing (ICIIP)*. 2025, pp. 175–182. DOI: 10.1109/ICIIP68302.2025.11346118.

[6] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. 3rd ed. Cambridge, MA: MIT Press, 2009. ISBN: 978-0262033848.

[7] David F. Crouse. "On Implementing 2D Rectangular Assignment Algorithms". In: *IEEE Transactions on Aerospace and Electronic Systems* 52.4 (2016), pp. 1679–1696. DOI: 10.1109/TAES.2016.140952.

[8] Kai Deng, Zexin Ti, Jiawei Xu, Jian Yang, and Jin Xie. *VGGT-Long: Chunk it, Loop it, Align it – Pushing VGGT's Limits on Kilometer-scale Long RGB Sequences*. 2025. arXiv: 2507.16443 [cs.CV]. URL: https://arxiv.org/abs/2507.16443.

[9] Michiel van Dijk, Tom Morley, Marie Luise Rau, and Yashar Saghai. "A meta-analysis of projected global food demand and population at risk of hunger for the period 2010–2050". In: *Nature Food* 2.7 (2021), pp. 494–501. DOI: 10.1038/s43016-021-00322-9.

[10] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale". In: *International Conference on Learning Representations (ICLR)*. 2021.

[11] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. "A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise". In: *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD)*. Portland, OR: AAAI Press, 1996, pp. 226–231.

[12] Martin A. Fischler and Robert C. Bolles. "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography". In: *Communications of the ACM* 24.6 (1981), pp. 381–395. DOI: 10.1145/358669.358692.

[13] Yulu Gao, Bohao Zhang, Zongheng Tang, Jitong Liao, Wenjun Wu, and Si Liu. "VGGT-Segmentor: Geometry-Enhanced Cross-View Segmentation". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2026, pp. 21690–21700.

[14] Jordi Gené-Mola, Ricardo Sanz-Cortiella, Joan R. Rosell-Polo, Josep-Ramon Morros, Javier Ruiz-Hidalgo, Verónica Vilaplana, and Eduard Gregorio. "Fruit detection and 3D location using instance segmentation neural networks and structure-from-motion photogrammetry". In: *Computers and Electronics in Agriculture* 169 (2020), p. 105165. ISSN: 0168-1699. DOI: 10.1016/j.compag.2019.105165.

[15] Nicolai Häni, Pravakar Roy, and Volkan Isler. "A comparative study of fruit detection and counting methods for yield mapping in apple orchards". In: *Journal of Field Robotics* 37.2 (Aug. 2019), pp. 263–282. ISSN: 1556-4967. DOI: 10.1002/rob.21902. URL: http://dx.doi.org/10.1002/rob.21902.

[16] Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. 2nd ed. Cambridge, UK: Cambridge University Press, 2004. ISBN: 978-0521540513. DOI: 10.1017/CBO9780511811685.

[17] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd ed. New York, NY: Springer, 2009. ISBN: 978-0-387-84857-0. DOI: 10.1007/978-0-387-84858-7.

[18] Xiangyu Hong, Linlin Qin, Chun Shi, and Gang Wu. "FruitGaussian: 3D Gaussian Splatting for Automated Fruit Counting in Natural Orchard". In: *Proceedings of the 44th Chinese Control Conference (CCC)*. 2025, pp. 8004–8009. DOI: 10.23919/CCC64809.2025.11179457.

[19] Wenhao Hu, Wenhao Chai, Shengyu Hao, Xiaotong Cui, Xuexiang Wen, Jenq-Neng Hwang, and Gaoang Wang. "Pointmap Association and Piecewise-Plane Constraint for Consistent and Compact 3D Gaussian Segmentation Field". In: *International Journal of Computer Vision* 134.6 (2026), p. 255. DOI: 10.1007/s11263-026-02821-3.

[20] E. M. B. M. Karunathilake, Anh Tuan Le, Seong Heo, Yong Suk Chung, and Sheikh Mansoor. "The Path to Smart Farming: Innovations and Opportunities in Precision Agriculture". In: *Agriculture* 13.8 (2023), p. 1593. DOI: 10.3390/agriculture13081593.

[21] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. "3D Gaussian Splatting for Real-Time Radiance Field Rendering". In: *ACM Transactions on Graphics* 42.4 (July 2023). URL: https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/.

[22] Harold W. Kuhn. "The Hungarian Method for the Assignment Problem". In: *Naval Research Logistics Quarterly* 2.1–2 (Mar. 1955), pp. 83–97. DOI: 10.1002/nav.3800020109.

[23] Vincent Leroy, Yohann Cabon, and Jerome Revaud. "Grounding Image Matching in 3D with MASt3R". In: *Proceedings of the European Conference on Computer Vision (ECCV)*. 2024.

[24] Fengze Li, Yangle Liu, Jieming Ma, Hai-Ning Liang, Yaochun Shen, Huangxiang Li, and Zhijing Wu. *CountingFruit: Language-Guided 3D Fruit Counting with Semantic Gaussian Splatting*. 2025. arXiv: 2506.01109 [cs.CV].

[25] Xu Liu, Steven W. Chen, Shreyas Aditya, Nivedha Sivakumar, Sandeep Dcunha, Chao Qu, Camillo J. Taylor, Jnaneshwar Das, and Vijay Kumar. "Robust Fruit Counting: Combining Deep Learning, Tracking, and Structure from Motion". In: *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2018, pp. 1045–1052. DOI: 10.1109/IROS.2018.8594239.

[26] Xu Liu, Steven W. Chen, Chenhao Liu, Shreyas S. Shivakumar, Jnaneshwar Das, Camillo J. Taylor, James Underwood, and Vijay Kumar. "Monocular Camera Based Fruit Counting and Mapping With Semantic Data Association". In: *IEEE Robotics and Automation Letters* 4.3 (2019), pp. 2296–2303. ISSN: 2377-3774. DOI: 10.1109/lra.2019.2901987. URL: http://dx.doi.org/10.1109/LRA.2019.2901987.

[27] Dominic Maggio, Hyungtae Lim, and Luca Carlone. "VGGT-SLAM: Dense RGB SLAM Optimized on the SL(4) Manifold". In: *Advances in Neural Information Processing Systems* 38 (2025).

[28] Leonardo Mariga. *pyRANSAC-3D*. https://github.com/leomariga/pyRANSAC-3D. 2022.

[29] Lukas Meyer, Andreas Gilson, Ute Schmid, and Marc Stamminger. "FruitNeRF: A Unified Neural Radiance Field based Fruit Counting Framework". In: *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2024, pp. 1–8. DOI: 10.1109/IROS58592.2024.10802065.

[30] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. "NeRF: representing scenes as neural radiance fields for view synthesis". In: *Commun. ACM* 65.1 (Dec. 2021), pp. 99–106. ISSN: 0001-0782. DOI: 10.1145/3503250. URL: https://doi.org/10.1145/3503250.

[31] Juan C. Miranda, Jordi Gené-Mola, Manuela Zude-Sasse, Nikos Tsoulias, Alexandre Escolà, Jaume Arnó, Joan R. Rosell-Polo, Ricardo Sanz-Cortiella, José A. Martínez-Casasnovas, and Eduard Gregorio. "Fruit sizing using AI: A review of methods and challenges". In: *Postharvest Biology and Technology* 206 (2023), p. 112587. ISSN: 0925-5214. DOI: 10.1016/j.postharvbio.2023.112587.

[32] James Munkres. "Algorithms for the Assignment and Transportation Problems". In: *Journal of the Society for Industrial and Applied Mathematics* 5.1 (1957), pp. 32–38. DOI: 10.1137/0105003.

[33] Maxime Oquab et al. "DINOv2: Learning Robust Visual Features without Supervision". In: *Transactions on Machine Learning Research* (2024). ISSN: 2835-8856.

[34] Fabian Pedregosa et al. "Scikit-learn: Machine Learning in Python". In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.

[35] René Ranftl, Alexey Bochkovskiy, and Vladlen Koltun. "Vision Transformers for Dense Prediction". In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2021.

[36] Johannes L. Schönberger and Jan-Michael Frahm. "Structure-from-Motion Revisited". In: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 4104–4113. DOI: 10.1109/CVPR.2016.445.

[37] Shinji Umeyama. "Least-Squares Estimation of Transformation Parameters Between Two Point Patterns". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 13.4 (1991), pp. 376–380. DOI: 10.1109/34.88573.

[38] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. "Attention Is All You Need". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. 2017.

[39] Pauli Virtanen et al. "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python". In: *Nature Methods* 17 (2020), pp. 261–272. DOI: 10.1038/s41592-019-0686-2.

[40] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. "VGGT: Visual Geometry Grounded Transformer". In: *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2025, pp. 5294–5306. DOI: 10.1109/CVPR52734.2025.00499.

[41] Shuzhe Wang, Vincent Leroy, Yohann Cabon, Boris Chidlovskii, and Jerome Revaud. "DUSt3R: Geometric 3D Vision Made Easy". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2024, pp. 20697–20709.

[42] Yifan Wang, Jianjun Zhou, Haoyi Zhu, Wenzheng Chang, Yang Zhou, Zizun Li, Junyi Chen, Jiangmiao Pang, Chunhua Shen, and Tong He. "$\pi^3$: Permutation-Equivariant Visual Geometry Learning". In: *arXiv preprint arXiv:2507.13347* (2025).

[43] Mi Yan, Jiazhao Zhang, Yan Zhu, and He Wang. "MaskClustering: View Consensus Based Mask Graph Clustering for Open-Vocabulary 3D Instance Segmentation". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2024, pp. 28274–28284.

[44] Jianing Yang, Alexander Sax, Kevin J. Liang, Mikael Henaff, Hao Tang, Ang Cao, Joyce Chai, Franziska Meier, and Matt Feiszli. "Fast3R: Towards 3D Reconstruction of 1000+ Images in One Forward Pass". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2025.

[45] Qian-Yi Zhou, Jaesik Park, and Vladlen Koltun. "Open3D: A Modern Library for 3D Data Processing". In: *arXiv preprint arXiv:1801.09847* (2018).

# Declaration

I certify that I have written this thesis without outside help and without using sources other than those indicated, and that the thesis has not been submitted in the same or similar form to any other examination authority and has not been accepted by them as part of an examination. All statements that have been taken over verbatim or in spirit are marked as such.

During the preparation of this work, AI technologies were used to assist in the writing process. Specifically, DeepL (DeepL SE, Cologne, Germany) was used to translate text from German into English, and Claude Opus (Anthropic PBC, San Francisco, California, USA) was used to assist with grammar, spelling, style consistency, and readability. After using these tools, the manuscript was carefully reviewed and the content was edited as needed. No tools or services were used to generate content, research results, analyses, or conclusions. The content of this thesis was entirely created by myself, and I ensured that all material presented reflects my original work and research. I take full responsibility for the content of the thesis.

I agree that the thesis may be published and that it may be referred to in scientific publications.

The Friedrich-Alexander-Universität Erlangen-Nürnberg, represented by the Professorship for Cognitive Computer Vision, is granted a (non-exclusive) right to use this work and the programs created in connection with it.

Erlangen, 01.10.2026

_____

(Alexander Link)